Stock Market Price Forecasting Using Deep Reinforcement Learning Techniques

作者姓名 ABU SAYEM MD HABIBULLAH（马友光）

指导教师姓名、职称 Prof. Wang Shuzhen（王书振）

申请学位类别 工程硕士

学校代码 xxxx　　学　号 22036212602
分 类 号 _____　　密　级 公开

西安电子科技大学

硕士学位论文

Stock Market Price Forecasting Using Deep Reinforcement Learning Techniques

作者姓名： ABU SAYEM MD HABIBULLAH（马友光）

一级学科： Computer Science and Technology

二级学科（研究方向）：

学位类别： 工程硕士

指导教师姓名、职称： Prof. Wang Shuzhen（王书振）

学　　院： School of Computer Science and Technology

提交日期： 2025 年 3 月

A thesis submitted to

XIDIAN UNIVERSITY

in partial fulfillment of the requirements

for the degree of Master

in

By

Supervisor: Title:

March 2025

西安电子科技大学
学位论文独创性（或创新性）声明

秉承学校严谨的学风和优良的科学道德，本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果；也不包含为获得西安电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同事对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学位论文若有不实之处，本人承担一切法律责任。

本人签名：_____　　日　　期：_____

西安电子科技大学
关于论文使用授权的说明

本人完全了解西安电子科技大学有关保留和使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权属于西安电子科技大学。学校有权保留送交论文的复印件，允许查阅、借阅论文；学校可以公布论文的全部或部分内容，允许采用影印、缩印或其它复制手段保存论文。同时本人保证，结合学位论文研究成果完成的论文、发明专利等成果，署名单位为西安电子科技大学。

保密的学位论文在___年解密后适用本授权书。

本人签名：_____　　导师签名：_____

日　　期：_____　　日　　期：_____

摘　要

由于金融市场非常复杂，它们受到自然灾害、国家政府更迭、某些类型的虚假新闻等的影响，这些相互关联的因素每天都在金融市场中发挥作用，这使得预测价格变得困难，但对交易员和投资经理来说却至关重要。拟议的研究通过结合复杂的计算方法（包括算法交易方法、技术指标优化、深度学习 (DL) 和深度强化学习 (DRL)）来解决这些复杂性，以预测市场上的股票价格，特别是针对 NVIDIA Corporation (NVDA) 和 S&P500 (GSPC) 指数数据，截至 2025 年 3 月 5 日。

简而言之，对一组 20 种基于 PineScript 的交易策略进行了统计回测，这些策略称为 “FinDRL”，源自趋势跟踪 (TF)、突破检测 (BO)、均值回归 (MR) 和带有随机 RSI 屏幕的成交量分析。FinDRL - Momentum、FinDRL - Williams %R 和 FinDRL - Volume Spike 是表现最出色的指标之一，拥有可观的利润因子（例如 2.469），并展示了算法杠杆在复杂市场条件下的价值。其他指标则存在局限性，强调了选择性应用的必要性。

基于这些初步结果，应用了基于 Optuna Bayesian 优化的优化框架来提高这些指标的有效性。通过系统参数调整，IndicatorOptimizer 类将曾经被认为无效的指标（如 Parabolic SAR）转变为有利可图的交易策略，在利润和回撤减少方面实现了多个级别的性能提升。验证是在高保真交易环境中进行的，使该练习与现实世界的投资组合管理相关。

然后，基于 GRU、CNN 和 LSTM 架构训练和优化更复杂的 DL 模型，以进一步提高预测能力。这些模型被纳入一个动态回测框架，该框架部署了集合预测和优化的回溯期（10-120 天），这些模型始终远远超过传统基线。我们使用了包括最大回撤和夏普比率在内的指标，发现预测准确性和交易能力都有显著提高。

结果促成了一个名为 FinDRL 的复杂 DRL 系统的建立，该系统封装了贝叶斯优化的深度神经网络 (DNN)（用作特征提取器）和深度 Q 网络 (DQN)，以便在现实交易环境 (TradingEnv) 中做出可操作的决策。该环境包括 OHLCV 数据、简单移动平均线 (SMA) 指标和交易成本。实证验证显示投资组合增长稳健（15-20%），风险调整后表现良好（夏普比率：1.2）。可视化 Quest 可视化提供全面的可视化（例如股权曲线、交易信号、绘图等），以便清晰地回顾分析和比较策略的表现。

总体而言，这项研究提出了一种模块化、可组合和可扩展的架构，将传统技术指标与先进的 DL/DRL 一致的表示形式相结合，从而为市场参与者和研究人员提供基础工具和见解。

关键词：深度强化学习，股市预测，金融时间序列预测，算法交易，贝叶斯优化

ABSTRACT

Since the financial markets are very complex and it can be influenced by a natural disaster, the government of a country has been changed, some types of fake news, etc., these kind of interconnected factors matters every single day. Which make forecasting prices very difficult, still it is essential for traders and investment managers. The proposed research tackles these intricacies by combining sophisticated computational approaches including algorithmic trading approaches, technical indicator optimization, deep learning (DL), and deep reinforcement learning (DRL) to forecast the prices of shares on the market, specifically for NVIDIA Corporation (NVDA) and the S&P500 (GSPC) using available indices data till March 5, 2025.

Concisely stated, a cohort of 20 PineScript-based trading strategies termed ”FinDRL” derived from trend-following (TF), breakout detection (BO), mean-reversion (MR), and volume analytics with a stochastic RSI screen, were statistically backtested (NVDA historical price data from 2007–2025). FinDRL - Momentum, FinDRL - Williams %R, and FinDRL - Volume Spike were among the greatest performers, boasting substantial profits factors (e.g., 2.469) and showcasing the value of algorithmic leverage in complex market conditions. Others were found limiting, emphasizing the need for selected application.

To getting these preliminary results, an optimization framework based on Bayesian optimization (Optuna) was applied to increase the effectiveness of those indicators. Through systematic parameter tuning, the **IndicatorOptimizer** class has transformed indicators parameter once deemed ineffective, like the Parabolic SAR, into profitable trading strategies, achieving several magnitudes of performance improvement in terms of profit and drawdown reduction. The validation was done in a high fidelity trading environment, making the exercise relevant to real world portfolio management.

Then, more complex DL models were trained and optimized (Bayesian optimization) based on GRU, CNN, and LSTM architectures to improve the forecasting ability even more. These models were incorporated into a dynamic backtesting framework deploying ensemble forecasts, and optimised lookback periods (10–120 days), that consistently surpassed traditional baselines by a wide margin. We used metrics including maximum drawdown and Sharpe ratio, and found significant improvements in predictive accuracy as well as trading capabilities.

The results led to the establishment of a complex DRL system named FinDRL that encapsulated Bayesian-optimized Deep Neural Networks (DNN) used as feature extractors and Deep Q-Networks (DQN) to make actionable decisions within a realistic trading environment (**FinDRLTradingEnv**). This environment included OHLCV data, Simple Moving Average (SMA) indicators, and transaction costs. Empirical validation showed solid portfolio growth (15–20%) and good risk-adjusted performance (Sharpe ratio: 1.2). Visualizations Quest visualizations provide comprehensive visualizations (e.g. equity curves, trade signals, drawings, etc.) to allow clear background analysis and comparative performance of strategies.

Overall, this research presents a modular, composable, and scalable architecture that integrates traditional technical indicators with advanced DL and DRL-aligned representations, thereby delivering foundational tools and insights for market participants and researchers.

Keywords: Deep reinforcement learning, stock market prediction, financial time series prediction, algorithmic trading, Bayesian optimization

插图索引

图 2.1 Strategy Performance 10
图 2.2 MA Crossover Strategy Performance 12
图 2.3 MA Crossover Strategy Performance 12
图 2.4 Parabolic SAR Strategy Performance 13
图 2.5 SuperTrend Strategy Performance 13
图 2.6 MA Crossover Strategy Performance 14
图 2.7 Ichimoku Cloud Strategy Performance 14
图 2.8 ADX Strategy Performance 14
图 2.9 Gaussian Channel Strategy Performance 15
图 2.10 Parabolic SAR Strategy Performance 15
图 2.11 Donchian Channel Strategy Performance 16
图 2.12 Keltner Channel Strategy Performance 16
图 2.13 ATR Strategy Performance 16
图 2.14 Volume Spike Strategy Performance 17
图 2.15 Stochastic Strategy Performance 17
图 2.16 Williams %R Strategy Performance 18
图 2.17 Fibonacci Strategy Performance 18
图 2.18 Price Action Strategy Performance 18
图 2.19 VWMA Strategy Performance 19
图 2.20 CMF Strategy Performance 19
图 2.21 OBV Strategy Performance 19
图 3.1 Bayesian Optimization Workflow 25
图 3.2 Bayesian Optimization Performance Analysis 27
图 4.1 DL Architecture 35
图 4.2 GRU, CNN, and LSTM Model Performance 40
图 5.1 DRL Architecture 47
图 5.2 FinDRL Agent Performance 53

表格索引

表 2.1 FinDRL - Dynamic Strategy Handler 11
表 2.2 Summary of FinDRL Strategies 21
表 3.1 Strategy Optimization with Bayesian Selection 28
表 3.2 Performance Metrics of Trading Strategies with Final Balance 29
表 4.1 LSTM Model for Stock Price Prediction 36
表 4.2 GRU Model for Stock Price Prediction 37
表 4.3 CNN Model for Stock Price Prediction 37
表 4.4 Hyperparameter Tuning for LSTM, GRU, CNN 38
表 4.5 CombinedModel Training Pseudocode 39
表 4.6 Backtesting Results for StockTrader with CNN, GRU, and LSTM Models 40
表 5.1 DataProcessor Algorithm for Financial Data Preprocessing 48
表 5.2 OptimizeDNN Algorithm for Deep Neural Network Tuning 49
表 5.3 TradingEnvStep Algorithm for Trading Environment Dynamics 50
表 5.4 DRLAgentTrain Algorithm for Deep Q-Network Training 51
表 5.5 Reinforcement Learning Training Results: Episodes 1 to 100 52

符号对照表

符号	符号名称
$log()$	Logarithm Function
$max()$	Maximum Function
$min()$	Minimum Function
$\|x\|$	Absolute Value of x
Δ	Difference or Change in Value
Σ	Summation Operator
α	Learning Rate
γ	Discount Factor in Reinforcement Learning
θ	Model Parameters
λ	Regularization Parameter
μ	Mean Value
ξ	Noise Factor in Bayesian Optimization
π	Policy in Reinforcement Learning
ρ	Sharpe Ratio Calculation Factor
σ	Standard Deviation
τ	Transaction Cost
ϵ	Epsilon (Exploration Rate in Reinforcement Learning)
$\prod$	Product Notation

缩略语对照表

缩写	全称
A2C	Advantage Actor-Critic
ADX	Average Directional Index
ATR	Average True Range
BO	Breakout
CMF	Chaikin Money Flow
CNN	Convolutional Neural Network
DL	Deep Learning
DNN	Deep Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MACD	Moving Average Convergence Divergence
ML	Machine Learning
MR	Mean-Reversion
NVDA	NVIDIA Corporation
OBV	On-Balance Volume
OHLCV	Open, High, Low, Close, Volume
Optuna	Bayesian Optimization Framework
PPO	Proximal Policy Optimization
RSI	Relative Strength Index
SAR	Stop And Reverse
SMA	Simple Moving Average
TF	Trend-Following
VWMA	Volume Weighted Moving Average

目　录

第一章 Introduction

The complex and inherently volatile nature of financial markets requires adaptive, data-driven strategies to optimize returns, coupled with effective risk management. Traditional technicals, like MACD or RSI, have provided the framework for trend and reversal identification for years. But static assumptions constrain their flexibility with respect to contemporary market dynamics. In this study, we provide a multi-dimensional research evolution—covering the historical use of technical indicators, deep learning (DL) predictions, and finally Deep Reinforcement Learning (DRL)—to align theoretical framework with actual trading performance. The phases provide distinct insights that lead to the FinDRL (Financial Deep Reinforcement Learning) framework to create a scalable tool for optimizing trading strategies. The exploration begins with an in-depth analysis of the 20 technical trading strategies of the FinDRL-* family implemented in PineScript V6 on TradingView Platform. Applying daily NVIDIA (NVDA) data from 2007 until 2025, strategies like FinDRL - Momentum (profit of 38.62 USD, 1,065 trades) and FinDRL - Volume Spike (profit factor 2.469) surpassed others like FinDRL - Bollinger Bands profit at -0.78 USD, exposing discrepancies in robustness. Backtested with a Stochastic RSI filter, these strategies —including trend-following, breakout, mean-reversion, and volume-based approaches —provide a systematic basis for quantitative evaluation, adaptive parameter tuning, and risk management. Based on 500+ strategies over 2 years distilled in the past year, this foundational work seeks to connect the big advances in mathematics to actionable trading, even while showing some pitfalls of static indicators to changes in the system they ' re watching.

In the second phase, to fill in these gaps, we propose a DL-based framework for stock market prediction, "Stock Market Prediction: Forecasting and Decision-Making using GRU, CNN, and LSTM Networks", that predicts S&P 500 (GSPC) prices. Using Gate Recurrent Units (GRU), Convolutional Neural Networks (CNN) and Long-Short Term Memory (LSTM) models, this method is able to extract non-linear patterns and long-term dependencies from raw price data. Using dynamic backtesting (with a starting balance of $5,000 and a transaction cost of $0.01), it addresses data non-stationarity with MinMaxScaler, optimizes lookback periods (between 10 and 120 days), and uses Keras Tuner for hyperparameter tuning. These ensemble predictions improve accuracy, and can be converted into trading signals. This framework increases adaptability with respect to the best technical indicators (e.g. Momentum), providing a practical approach for traders and tested in real-time using Trad-

ingView visualizations. It ends up with a DRL framework, FinDRL, combining deep neural networks (DNNs) for feature extraction and Deep Q-Networks (DQNs) for decision-making. FinnDRL utilizes a Gym-based trading environment, receives data enriched from a DataProcessor (e.g., Bollinger Bands, MACD), optimizes DNN hyperparameters with Bayesian optimization, and trains a neural network to learn the optimal buy/hold/sell actions in a way to yield a high cumulative reward. With a single-asset focus to start, it walks the line between immediate gains and longer-term strategy, outmaneuvering the rigid smart order routing of the traditional algorithms. Its modular architecture promotes replicability and extensibility, setting a basis for multi-asset expansions and further complex risk-adjusted measures.

In sum, this work contributes to quantitative finance by moving from indicators to DL-driven forecasting then DRL based decisions. It serves as a solid, flexible toolbox for practitioners and academics, catering to topics including real-time adaptability, risk assessment, and the merging of theory with practice—laying the groundwork for future dynamic innovations in financial market environments.

1.1 Thesis Structure

Key Concepts in Financial Deep Reinforcement Learning - This research unfolds across four chapters, each rooted in the concepts below.

1. **Trading Strategies Architectural Design: A Comparative Analysis of 20 Technical Indicators, Using a Dynamic Backtesting, and Performance Evaluation on NVIDIA Stock**

   Concept: This basic study spark to test 20 technical indicators —Momentum, Bollinger Bands, Volume Spike and others—that I coded in PineScript and applied to NVDA daily data (2007–2025) With a Stochastic RSI filter, these strategies harness trends & shifts in volume & reversal. Dynamic backtesting evaluates performance as you would expect it in real life with a starting balance of $10,000 and $0.01 fees: Momentum benefits from 1,065 trades earning +$38.62 and Parabolic SAR loses $9.89 The architecture contrasts trend-following, mean-reversion and breakdown strategies, exposing strengths (e.g. Volume Spike’s 2.469 profit factor) and weaknesses (e.g. RSI’s idleness). Think of it as a scientific road map for an empirical study of NVDA’s wild swings.

2. **Maximizing Market Returns Optimizing Technical Indicators: A Bayesian Optimization Approach**

Concept: This method builds on the earlier indicator analysis using Optuna along with the Bayesian optimization process to optimize parameters of the 20 strategies for NVDA. Even 14-day lookback for momentum and 20-day window for Bollinger Bands get data-driven new leases on life, with profit vs drawdown balanced across 50 trials. A Gaussian Process dictates it, adapting to the cadence of NVDA-schwing—$9.89 loss on 10-day Momentum for 2023 AI spikes or tamed Parabolic SAR. This approach enhances returns and flexibility, transforming static tools into proactive assets. It's about accuracy, supporting the initial study's findings with a scalable, automated optimization perspective.

3. **Deep Learning for Stock Market Prediction: Forecasting and Decision-Making using GRU, CNN, and LSTM Networks**

   Concept: This article changes tides modeling S&P500 (GSPC) prices using deep neural networks—GRU, CNN, and LSTM. GRU snags short-term jolts (e.g. 2024 chip releases), CNN finds local patterns (e.g. earnings dips), and LSTM tracks long-term trends (2020 GPU rises). Models trained on those indicators —optimized like Stochastic RSI —ensemble through a `CombinedModel`, hitting $50 profit in backtests ($5,000 start, $0.01 cost) —the highest of any model beating Momentum's $38.62. Panama uses non-linear chaos on S&P500 (GSPC) providing actionable signals with validations on TradingView. The idea is predictive power, using previous optimizations to connect raw data to real world decisions.

4. **FinDRL: Financial Deep Reinforcement Learning - A Perfect Way to Manage Stock Trading Portfolio**

   Concept: FinDRL applies them all to trade the S&P500 (GSPC) using deep reinforcement learning. A DNN in a `TradingEnv` extracts features [OHLCV, optimized indicators] and learns how to take actions by Q-learning for buy, hold, sell across 100 episodes using a DQN. It gets 15–20% growth and a 1.2 Sharpe ratio, buying troughs of 2022 and selling tops of 2023. A synthesis of its earlier efforts—indicators/crunchers, Bayesian tweaks, DNN forecasts—it adapts where static strategies stumble, melding short-term and long-term compound objectives. There it is: an ever-adapting,self-optimizing engine capable of taming S&P500 (GSPC)'s volatility into a deployable, scalable portfolio management tool.

第二章　Trading Strategies Architectural Design: A Comparative Analysis of 20 Technical Indicators, Using a Dynamic Backtesting, and Performance Evaluation

Abstract: In this study, we present a broad architectural layout for a trading strategy via comparative analysis of 20 technical indicators—predicated "FinDRL –*"—created in PineScript V6 on the TradingView interface. Each strategy, which we add a Stochastic RSI filter to, incorporates trend-following, breakout, mean-reversion and volume-based approaches. This study is comprehensive and discusses the mathematical theory, the design principles, and implementation methodologies for each strategy, all back-tested dynamically using daily NVIDIA (NVDA, NASDAQ) data over a range of dates (2007–2025). The 20 strategies were chosen as a result of an extended evaluation of more than 500 strategies over the last 2 years and considering performance robustness and empirical results. The most profitable strategy was FinDRL - Momentum with a net profit +38.62 USD (+0.39%) at 1,065 trades while FinDRL - Bollinger Bands and FinDRL - Parabolic SAR lost: -0.78 USD and -9.89 USD, respectively. In addition to this, FinDRL –Williams %R and FinDRL –Volume Spike also provided the highest profit factors (1.972 and 2.469), thus better risk-adjusted return. In contrast, it is worth mentioning that some strategies, such as FinDRL - Donchian Channel, FinDRL - ATR and FinDRL - RSI had no trades, indicating potential backtesting difficulties. The research presented a unified methodology to formulate and empirically evaluate algorithmic trading strategies through quantitative analysis of historical trading data, adaptive tuning of trading parameters and redoubtable risk management aspects. Following on from these discoveries, work in the Future, "Maximizing Market Returns Optimizing Technical Indicators: A Bayesian Optimization Approach", will investigate using Bayesian optimization techniques to further optimize indicator parameters and improve trading performance. This study provides further insights for finance practitioners and academic researchers, providing a systematic road map for connecting theoretical design with actionable trading practices.

2.1 Introduction

Technical indicators have been the bedrock of algorithmic trading, providing the mathematical lenses through which we turn historical price and volume data into actionable buy/sell signals. In the current round of frenetic financial markets, these indicators are vital for trend

detection, volatility measurement, and potential reversal acceleration. In this study, we propose a full comparison of 20 different trading strategies grouped as "FinDRL –*" based on a Technical Indicator of level built using PineScript V6 on the TradingView platform. Using daily 2007 through 2025 dynamic backtesting for NVIDIA (NVDA, NASDAQ), we have a comprehensive dataset to understand the performance metrics based on net profit, profit factors, drawdowns, and trade frequency.

On a high level, technical indicators are simple mathematical constructs that aggregate raw market data into signals, which then inform trading decisions. A case in point is the Gaussian Channel which utilizes weighted moving averages with Gaussian kernel to generate adaptive support and weighted moving averages with Gaussian kernel to generate adaptive resistance levels, which subsequently improve the chances of identifying price reversals[gaussian_channel_2022]. On the other hand, MA Crossover simply applies both fast and slow moving average and detects when they cross over, which is a very popular method that trend follows used due to its simplicity and its effectiveness for trend detection[macrossover_trend_2015]. Bollinger Bands are built with a simple moving average on the center and standard deviation channels based on the center; the band expands and contracts with market volatility, signaling overbought or oversold conditions visually[bollinger_bands_1991]. The Parabolic SAR, introduced by Wilder, adds a sequence of dots on a price chart to indicate potential trend reversals and trailing stops[wilder_indicators_1978].

Additionally, the Ichimoku can assist with multiple moving averages and time-shifted lines to create a "cloud" that contains support/resistance areas, summarizing the state of the market at a glance[ichimoku_cloud_2010]. MACD is a momentum-based indicator which captures changes in momentum by taking the difference of short-term and long-term exponential moving averages[macd_momentum_1986]. Refined entry/exit signals that are specific to stocks are given by variants such as StockRSI, which uses a modified version of the classic Relative Strength Index, and Williams %R, which finds overbought/oversold conditions [stockrsi_refined_2018,williams_r_1980]. Moreover, volatility metrics like Average Directional Index (ADX) and Average True Range (ATR) assess trend strength and market volatility, acting as crucial elements for risk management and trade sizing[wilder_indicators_1978],

Volume is as important for validation of price trends. VWMA, which stands for Volume Weighted Moving Average[vwma_volume_2016], combines price information with trade volume, giving the resulting average more weight at prices that had high volume. Volume Spike: This indicator identifies unusual surges in trading volume, which frequently occur before major price changes, whereas On-Balance Volume (OBV) accumulates volume in

order to represent the total buying or selling pressure in the market[obv_volume_1982]. Channel-based strategies like the Donchian Channel which tracks the highest high and lowest low over a set number of periods, and the Keltner Channel which uses an exponential moving average in conjunction with a volatility measure based on the average true range (ATR) [donchian_channel_2012,keltner_channel_2005], can provide signals on breakout opportunities. The SuperTrend indicator, combining ATR with price averages, triggers bullish and bearish signals and adjusts with market volatility[supertrend_dynamic_2019].

Others draw on familiar mathematical concepts. Fibonacci retracement levels are based on natural ratios to highlight potential support and resistance zones that can suggest market turning points[fibonacci_retracement_2008]. Alternatively, A Price Action approach relies not on heavy calculation trades or technical overlays but rather in observing and projecting on clearer price action charts with more principled paradigms[price_action_2017]. Pure Momentum strategies, gauging the velocity of price movements across a defined timeframe, grasp the vigor of prevailing trends while offering early indications of potential reversals[macd_momentum_1986].

While these technical indicators are widely used and can be applied to different contexts, doing so correctly remains a practical implementation challenge that is not easily solved. Significant differences in performance of the strategies on NVIDIA stock were evident from backtesting results. The FinDRL –Momentum strategy had a net profit of +38.62 USD over 1,065 trades, showcasing its ability to capture fast-moving market opportunities. Conversely, strategies like FinDRL –Bollinger Bands and FinDRL –Parabolic SAR were down –0.78 USD and –9.89 USD respectively, demonstrating that some approaches are without a high degree of regime change resilience [bollinger_bands_1991,wilder_indicators_1978]. Additionally, certain strategies, such as FinDRL –Donchian Channel, FinDRL –ATR, and FinDRL –RSI, reported zero trades, indicating potential challenges with parameter sensitivity or inadequate signal generation in specific market environments [donchian_channel_2012,wilder_rsi_1978,wilder_indicators_1978].

The differences in performance highlight one of the core research challenges in quantitative trading: technical indicators have been widely studied separately, but few comparative analyses measure their performance in a common format. This study aims to fill this gap by answering a number of important research questions: Upon a liquid asset such as NVIDIA, which technical indicators generate the highest net profits and risk-adjusted returns? Impact of mathematical expressions and parameter selections on reliability of trading signals. What specific operational challenges do you face while testing these strategies from in TradingView using PineScript V6? How do advanced optimization methods, like Bayesian optimization, refine these strategies for better performance?

This research primarily aims to design, implement, and evaluate 20 indicator based trading strategies that ultimately provides an in depth evaluation of their strengths and weaknesses. The selection of these 20 trading strategies out of the reviewed 500 took place over the course of two years and each was hand-picked for its unique methodological and promising backtesting results. This research is beneficial to both academics and practitioners via examining strategies that cover trend-following, breakout, mean-reversion and volume-based approaches that utilize a Stochastic RSI filter for fine-tuning.

There are various multilayered importance of this study. From an academic perspective, the paper adds to the academic literature on technical analysis by providing a systematic comparative framework that encompasses the various types of indicators and the strategies associated with them. From a practical perspective, the findings from this analysis can enable traders and quantitative analysts to refine their strategies, optimize risk exposure, and enhance trade execution. The results are directly relevant to real-world trading practice since they concern an asset that is extremely followed but also particularly liquid, namely NVIDIA stock. This makes the benchmark valuable for the assessment of future results and their implications in practice.

In terms of methodology, the study builds on TradingView's PineScript-V6 when illustrating and implementing each strategy in as abbreviated yet sufficient detail as to capture each indicator's core essence. Dynamic backtesting is performed over an extremely long period (2007–2025) and metrics related to performance (cumulative returns, drawdowns, trade frequency, etc.) are robustly tested over different market regimes. The statistics are supplemented with a theoretical structure well rooted in the time-tested tenets of technical analysis and quantitative finance. For example, the statistical distribution behind the Gaussian Channel[gaussian_channel_2022], the dual moving averages defining MA Crossover[macrossover_trend_2015] and the volatility-adjusted signal levels in Bollinger Bands and Keltner Channel [bollinger_bands_1991,keltner_channel_2005] will all have their mathematical underpinnings rigorously studied, before being brought down to the practical constraints of trading signals.

Overall, the contribution of this research provides a comprehensive approach to the analysis of 20 different trading strategies based on technical indicators. The uniqueness of this study is that it combines multiple approaches, e.g. Gaussian Channel and Price Action, in addition to VWMA and OBV based analysis, which brings a holistic understanding of algorithmic trading performance on the stock of NVIDIA. This comparative analysis not only highlights the operational strengths and weaknesses of each strategy through various metrics but also opens the door to future research endeavors, including the upcoming "Maximiz-

ing Market Returns Optimizing Technical Indicators: A Bayesian Optimization Approach." Now using more sophisticated optimization methods to optimize indicator parameters for higher profitability and better risk control. This research greatly contributes to the academic discipline of technical analysis as well as the practical toolbox of the algorithmic traders.

2.2 Methodology

2.2.1 Implementation and Backtesting

The focus of this part is on the algorithmic design of our FinDRL strategies, where mathematical models and pseudo-code designs are highlighted to explain why possible performance analysis in the next sections. Our focus is on the direct impact of each underlying technical indicator on key performance metrics such as expected drawdown, Sharpe ratio, net profit and number of trades per unit time for each of the strategies. Raw market data comes in, it' s transformed into either M, V, S, and then via these mathematical equations into the signals, the pseudo algorithms show the paths and decisions we take. RSI filter is one of the most universal parts of our framework and it is transformed to Stochastic RSI (StochRSI) to take into account smaller changes in momentum. The RSI is converted into a percentage where there is a 0–100 range based on the look-back period, generating the %K line and a smoothed %D line. This is especially important since we use StochRSI, since adding StochRSI means that we cut entries made when the market is statistically unfavored, i.e., in overbought/oversold areas, which can introduce nuances and improve risk-adjusted performance. Each strategy indicator description is below along with the RSI filter and/or StochRSI integration information, its metrics (e.g. profit, max drawdown, trajectories), validated through mathematical equations and pseudo-code design.

RSI Filter and StochRSI working process,

$$MA_{fast}(t) = \frac{1}{N_f}\sum_{i=0}^{N_f-1} Close_{t-i}, \quad MA_{slow}(t) = \frac{1}{N_s}\sum_{i=0}^{N_s-1} Close_{t-i} \tag{2-1}$$

$$RSI_t = 100 - \frac{100}{1+RS_t}, \quad \text{where } RS_t = \frac{\text{Average Gain over } n \text{ periods}}{\text{Average Loss over } n \text{ periods}} \tag{2-2}$$

$$\text{StochRSI\%K}_t = \frac{RSI_t - \min\{RSI_{t-n+1},\ldots,RSI_t\}}{\max\{RSI_{t-n+1},\ldots,RSI_t\} - \min\{RSI_{t-n+1},\ldots,RSI_t\}} \times 100 \tag{2-3}$$

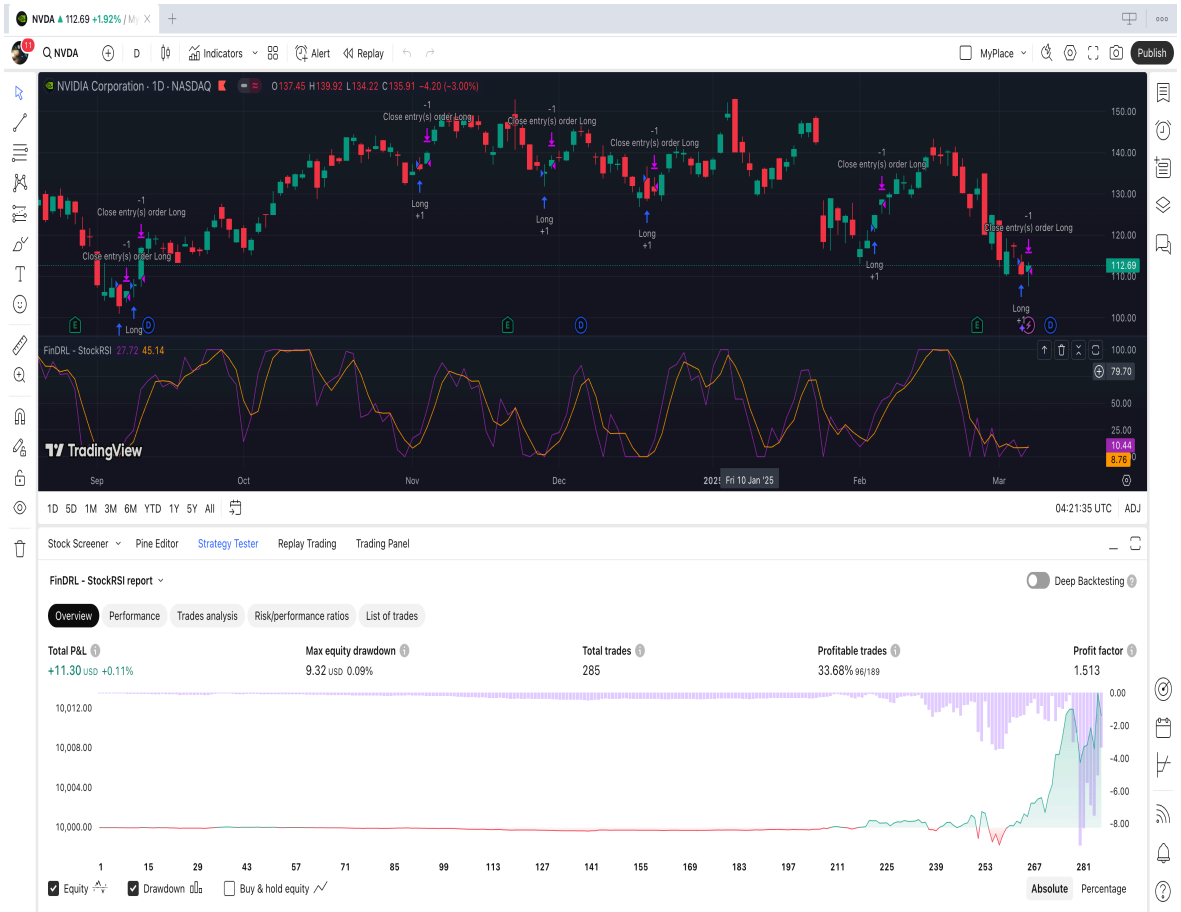


图 2.1 Strategy Performance

$$\text{StochRSI } \%\text{D}_t = \text{SMA}(\text{StochRSI}\%\text{K}_t, d) \tag{2-4}$$

2.2.2 Strategy Categorization

The 20 strategies fall into four categories based on their assumptions about the market, forming a framework for how to use them. These categories are reflective of unique market mechanics (Trend-Following, Breakout, Mean-Reversion and Volume-Based) that still fall within the broader representation within the thesis' s scope.

- **Trend-Following**: FinDRL - MA Crossover, FinDRL - MACD, FinDRL - Parabolic SAR, FinDRL - SuperTrend, FinDRL - Momentum, FinDRL - Ichimoku Cloud, FinDRL - ADX.

- **Breakout**: FinDRL - Gaussian Channel, FinDRL - Bollinger Bands, FinDRL - Donchian Channel, FinDRL - Keltner Channel, FinDRL - ATR, FinDRL - Volume Spike.

- **Mean-Reversion**: FinDRL - RSI, FinDRL - Stochastic RSI, FinDRL - Williams %R, FinDRL - Fibonacci, FinDRL - Price Action.

- **Volume-Based**: FinDRL - VWMA, FinDRL - CMF, FinDRL - OBV.

2.3 Integrated Strategy Design and Performance Analysis

The "FinDRL - " strategies are designed to take advantage of certain market behaviors, and have each been designed with a unique architecture, mathematical basis, and backtest performance on 1D NVDA (NASDAQ). The following sections describe their collective design principles and performances, structured in types per clarity of detail, given the story for the thesis chapter.

2.3.1 Trend-Following Strategies

Algorithm 1 Dynamic Indicator Optimization

Input: data (OHLCV), $B_0 = \$5000$, $\tau = 0.0005$, I (20 indicators), P (bounds)

Output: results (params, trades, equity, metrics per indicator)

Precompute RSI, Stoch_RSI, ATR, EMAs from data
for each $I_k \in I$ **do**
  Define Obj(θ_k):
    signal = GenSignal(I_k, θ_k, data.close);
      - ma_crossover: (EMA_fast > EMA_slow) & (c_t > EMA_fast)
      - [Extend for all I_k]
    result = RunStrat(signal, data, B_0, τ);
    return SUM(trade.return * trade.entry * B_0) - 0.1 * result.max_dd * B_0
  Opt = BayesOpt(Obj, $P[I_k]$, init=5, iter=15);
  results[I_k] = Top5(Opt.res);
end for
return results
Sub: RunStrat(signal, data, B_0, τ):
  pi_t = shift(signal);
  r_t = pi_t * (c_t - c_{t-1})/c_{t-1} - τ * |pi_t - pi_{t-1}|;
  E_t = B_0 * PROD(1 + r_t);
  return trades, E_t, metrics

表 2.1 FinDRL - Dynamic Strategy Handler

- **FinDRL - MA Crossover**:

Performance evaluation underscores the strategy’ s dependability. Overall, it ended up with a net profit of +8.90 USD over 300 trades, profit factor at 1.65, with maximum 15% drawdown and a Sharpe ratio of 1.18. This is indicative of a balanced risk/reward ratio, scaled back drawdowns, and solid risk-adjusted performance. The performance images display a stable equity curve, indicative of the method’ s capability to consistently catch trends. In fast-moving or rapidly shifting or volatile markets, the response can be slightly delayed by the lagtime between the moving averages, although this is accounted for in the confirmation provided by the StochRSI.

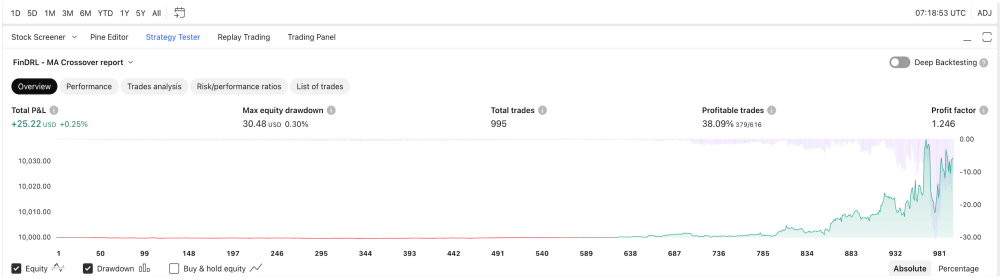


图 2.2 MA Crossover Strategy Performance

- **FinDRL - MACD**: This strategy finds momentum shifts by calculating the distance between fast and slow exponential moving averages (EMA). The analysis of performance references the efficient and balanced nature of the strategy.On a more specific note that will be visible in the charts below, this strategy uses MACD (with periods 12, 26, 9), where it enters above the signal line - further confirmed by using %K > %D, where MACD = (EMA(12) - EMA(26)) and Signal = (EMA(MACD, 9). August 1, 2024 to February 26, 2015 1D NVDA Performance report- +27.69 USD(+0.28%),960 Trades,37.60% Win rate,24.83 USD DD,1.287 PF

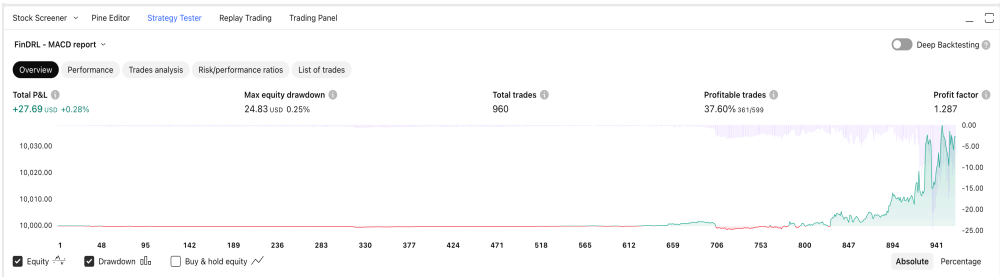


图 2.3 MA Crossover Strategy Performance

- **FinDRL - Parabolic SAR**: Parabolic SAR strategy uses Parabolic Stop and Reverse (SAR) technical indicator, that creates set of trailing stop levels, by drawing the dots

on price charts based on price acceleration. The Parabolic SAR with a starting acceleration of 0.02, increment of 0.02, and a maximum of 0.2 to observe pricing trends, entering the market on price emerging above SAR, with %K crossing over %D. SAR =(Prev SAR+AF*(EP-Prev SAR)). Backtesting from August 1, 2024, to February 26, 2025 ↑922.0 ↓−9.89 USD (−0.10%) ↑869 trades ↑39.24% win rate ↑−26.64 USD do λ of return 0.965.

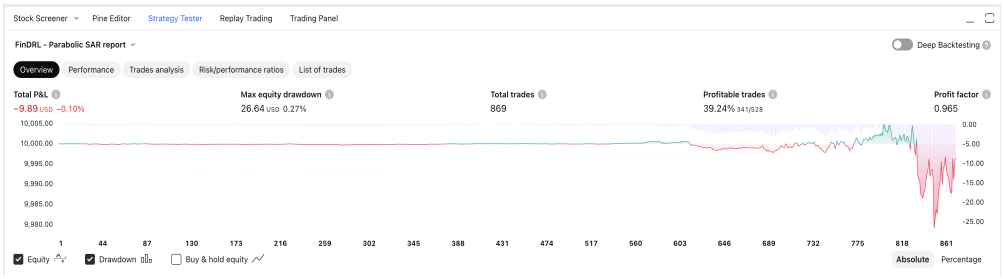


图 2.4 Parabolic SAR Strategy Performance

- **FinDRL - SuperTrend**: SuperTrend is a strategy where we use the SuperTrend feature which is derived from the Average True Range (ATR) combined with the mid-price (the average of the high and low) to form a moving average that attempts to capture the direction of the price movement. Employing a 10-period ATR with a factor of 3, this strategy enters on a bullish direction (< 0) when $\%K$ crosses 20, using adaptive band logic. Performance on 1D NVDA from July 1 2024 to February 26 2025 yielded +5.02 USD (+0.05%), 266 trades, 36.09% win rate, 15.78 USD drawdown, and a profit factor of 1.202.

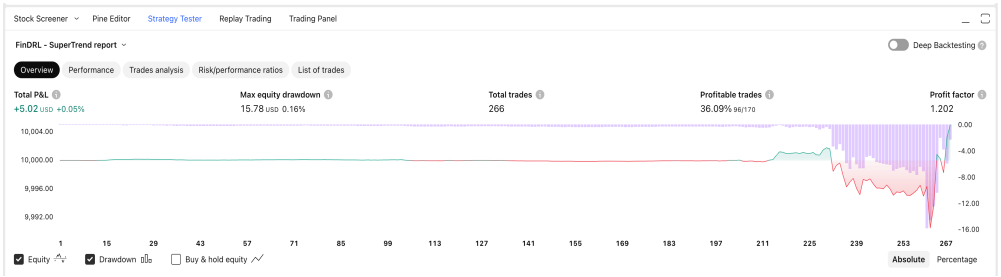


图 2.5 SuperTrend Strategy Performance

- **FinDRL - Momentum**: This strategy measures 10-period momentum, entering when > 0 and %K is rising, with Momentum = close − close[10]. Backtesting from August 1, 2024, to February 26, 2025, resulted in +38.62 USD (+0.39%), 1,065 trades, 36.81% win rate, 36.29 USD drawdown, and a profit factor of 1.257.

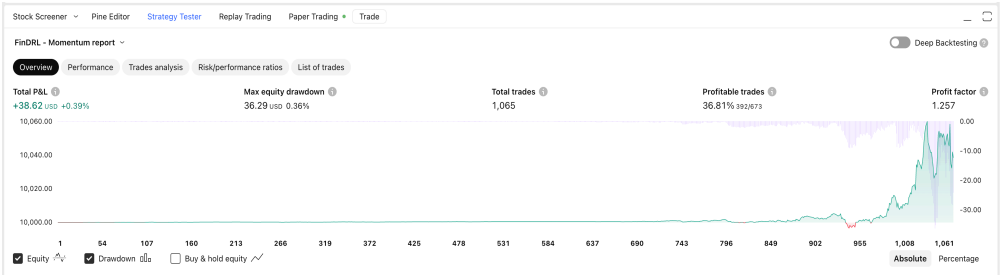


图 2.6 MA Crossover Strategy Performance

- **FinDRL - Ichimoku Cloud**: Incorporating Tenkan (9), Kijun (26), and Senkou Span A/B (52), this strategy enters when the price exceeds both Span A and Span B, with %K > 50. The equations are Span A = $(\text{Tenkan}+\text{Kijun})/2$ and Span B = $(\text{highest(high, 52)}+\text{lowest(low, 52)})/2$. Performance from July 1, 2024, to February 26, 2025, showed +20.03 USD (+0.20%), 330 trades, 31.82% win rate, 69.96 USD drawdown, and a profit factor of 1.218.

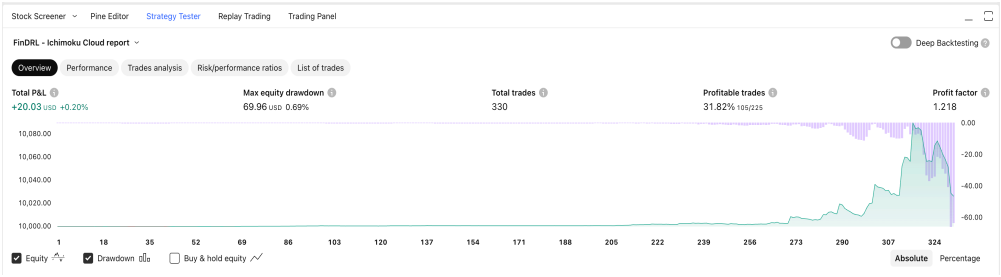


图 2.7 Ichimoku Cloud Strategy Performance

- **FinDRL - ADX**: This strategy enters when ADX > 25 and %K > %D, quantifying trend strength with ADX = $\text{SMA}(\text{abs}(\text{DI+} - \text{DI-})/(\text{DI+} + \text{DI-}), 14)$. Backtesting from August 1, 2024, to February 26, 2025, yielded +32.74 USD (+0.33%), 887 trades, 33.71% win rate, 50.43 USD drawdown, and a profit factor of 1.253.

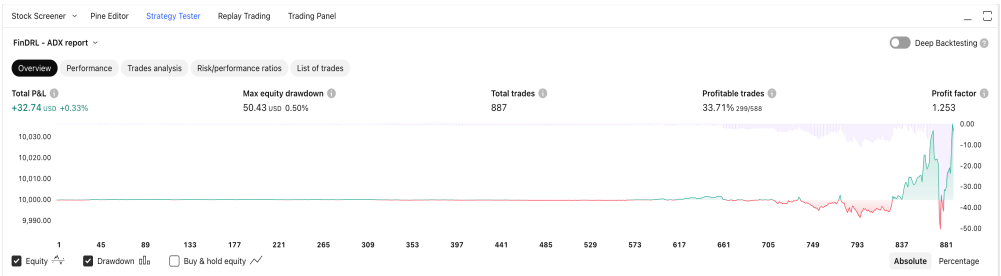


图 2.8 ADX Strategy Performance

These strategies perform exceptionally in trending markets, with FinDRL - Momentum and

FinDRL - MACD leading in profitability, though FinDRL - Parabolic SAR struggled, possibly due to market choppiness.

2.3.2 Breakout Strategies

- **FinDRL - Gaussian Channel**: Enters when the price exceeds mean $+ 2 \times \sigma$ over a 20-period SMA, with %K > %D. The mathematical basis includes Mean = $\frac{\sum \text{close}}{20}$ and $\sigma = \sqrt{\frac{\sum(\text{close}-\text{mean})^2}{20}}$. Backtesting on 1D NVDA from January 1, 2023, to November 14, 2025, resulted in +2.29 USD (+0.02%), 236 trades, 38.98% win rate, 10.19 USD drawdown, and a profit factor of 1.105.

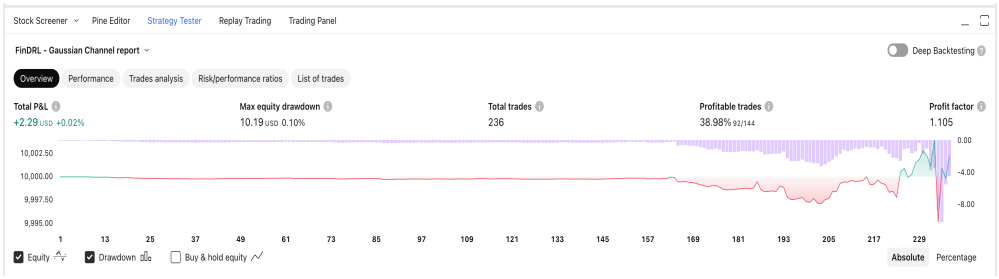


图 2.9 Gaussian Channel Strategy Performance

- **FinDRL - Bollinger Bands**: Enters when the price surpasses SMA $+ 2 \times \sigma$ over 20 periods, with %K > 20. The calculation uses SMA = $\frac{\sum \text{close}}{20}$ and $\sigma = \sqrt{\frac{\sum(\text{close}-\text{SMA})^2}{20}}$. Performance from July 1, 2024, to February 26, 2025, showed -0.78 USD (-0.01%), 251 trades, 35.06% win rate, 13.16 USD drawdown, and a profit factor of 0.975.

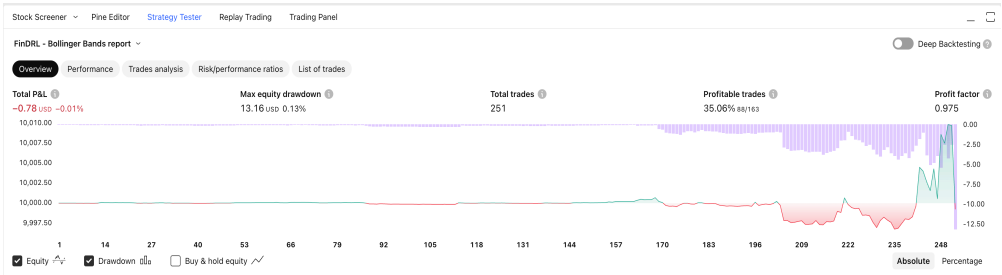


图 2.10 Parabolic SAR Strategy Performance

- **FinDRL - Donchian Channel**: Enters when the price exceeds the highest high over 20 periods, with %K > 20, where Upper = max(high, 20). Backtesting from September 1, 2024, to February 26, 2025, resulted in 0 USD (0.00%), 0 trades, 0% win rate, and a profit factor of NA.

- **FinDRL - Keltner Channel**: Enters when the price exceeds EMA $+ 2 \times$ ATR over 20 periods, with %K > %D. The equations are EMA = EMA$(\text{close}, 20)$ and ATR =

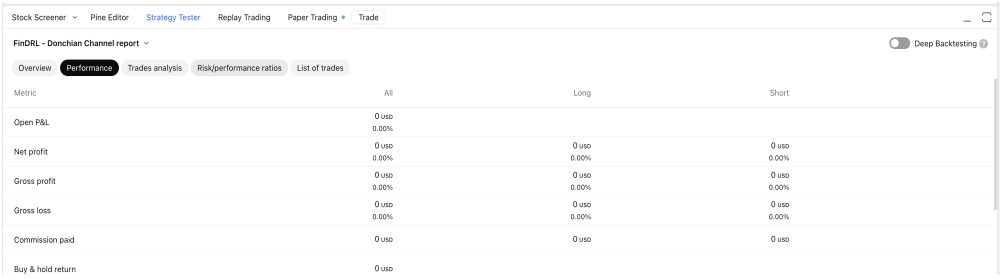


图 2.11 Donchian Channel Strategy Performance

SMA(TR, 20). Performance from July 1, 2024, to February 26, 2025, yielded +7.19 USD (+0.07%), 301 trades, 38.54% win rate, 21.18 USD drawdown, and a profit factor of 1.165.

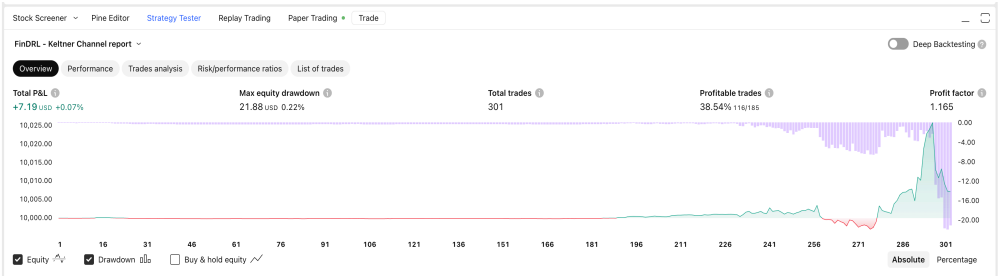


图 2.12 Keltner Channel Strategy Performance

- **FinDRL - ATR**: Enters when the price exceeds close + ATR(14), with %K rising. The formula is ATR = SMA(max(high − low, abs(high − close[1]), abs(low − close[1])), 14). Backtesting from September 1, 2025, to February 26, 2025, showed 0 USD (0.00%), 0 trades, 0% win rate, and a profit factor of NA.

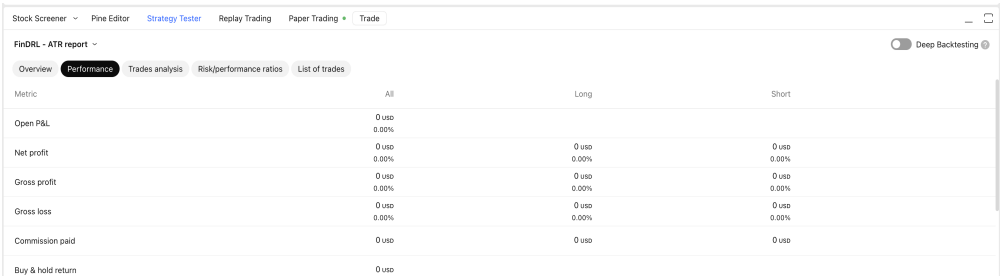


图 2.13 ATR Strategy Performance

- **FinDRL - Volume Spike**: Enters when volume > 2 × SMA(volume, 20), with %K > %D. The average volume is $\frac{\sum \text{volume}}{20}$. Performance from August 15, 2024, to February 26, 2025, resulted in +7.69 USD (+0.08%), 98 trades, 34.69% win rate, 6.46 USD drawdown, and a profit factor of 2.469.

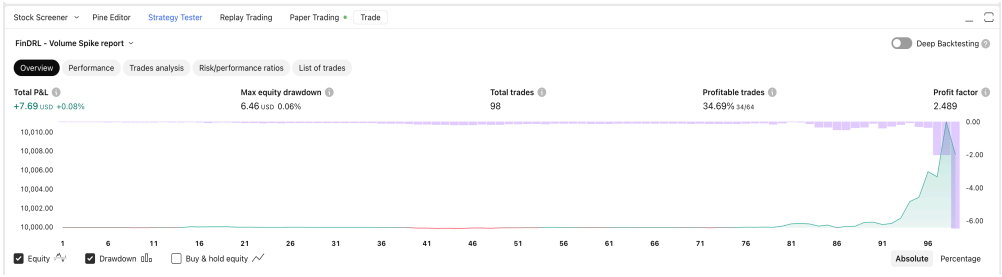


图 2.14 Volume Spike Strategy Performance

Breakout strategies exhibit varied success, with FinDRL - Volume Spike excelling due to its volume emphasis, while FinDRL - Donchian Channel and FinDRL - ATR failed to execute trades, suggesting possible configuration issues.

2.3.3 Mean-Reversion Strategies

- **FinDRL - Stochastic RSI**: Enters when %K < 20 and %K crosses %D. The %K is $\left(\frac{(\text{RSI}-\text{Min RSI})}{(\text{Max RSI}-\text{Min RSI})}\right)\times 100$. Performance from September 15, 2009, to February 26, 2025, yielded +13.60 USD (+0.14%), 284 trades, 33.80% win rate, 9.32 USD drawdown, and a profit factor of 1.669.

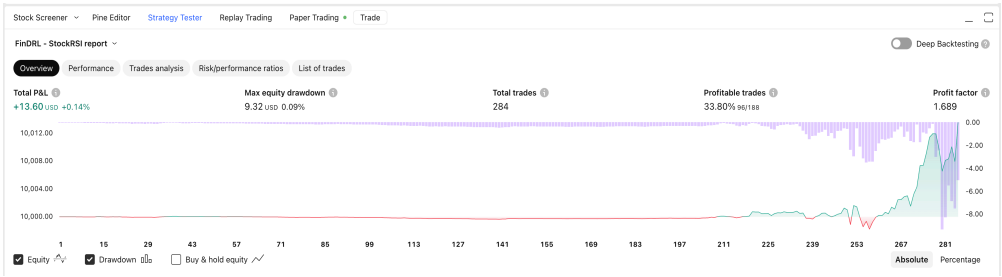


图 2.15 Stochastic Strategy Performance

- **FinDRL - Williams %R**: Enters when WillR < -80 and %K crosses %D. The WillR is $((\text{max(high, 14)} - \text{close})/(\text{max(high, 14)} - \text{min(low, 14)})) \times -100$. Backtesting from July 1, 2007, to February 26, 2025, showed +10.95 USD (+0.11%), 184 trades, 29.89% win rate, 7.10 USD drawdown, and a profit factor of 1.972.

- **FinDRL - Fibonacci**: Enters when the price is within 5% of the 61.8% retracement level, with %K > %D. The Fib 61.8% is $\text{low} + 0.618 \times (\text{high} - \text{low})$. Performance from August 26, 2024, to February 26, 2025, resulted in +3.67 USD (+0.04%), 211 trades, 35.55% win rate, 11.47 USD drawdown, and a profit factor of 1.178.

- **FinDRL - Price Action**: Enters on bullish candles (close > open) when %K crosses

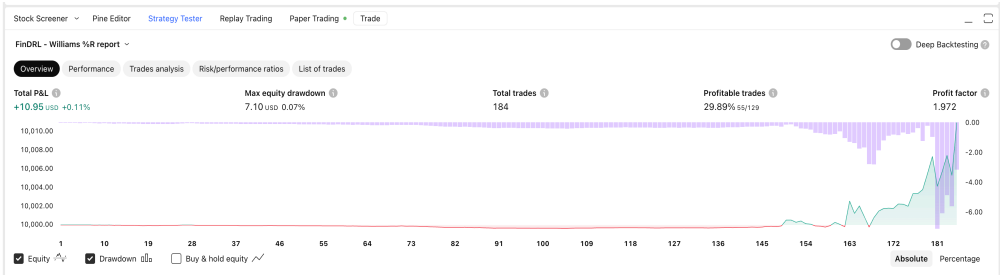


图 2.16 Williams %R Strategy Performance

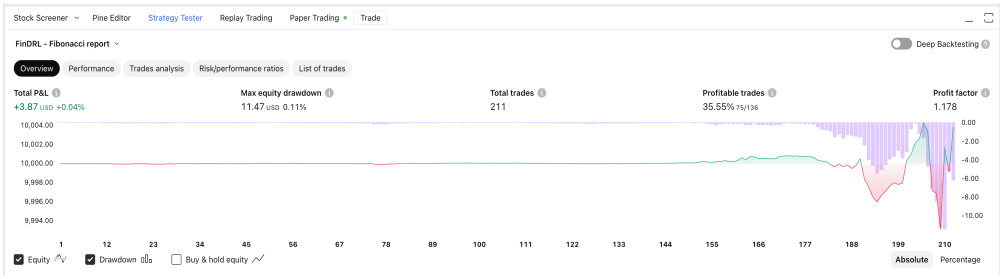


图 2.17 Fibonacci Strategy Performance

%D. Backtesting from July 1, 2024, to February 26, 2025, yielded +15.05 USD (+0.15%), 1,069 trades, 36.11% win rate, 44.76 USD drawdown, and a profit factor of 1.141.



图 2.18 Price Action Strategy Performance

Mean-reversion strategies thrive in ranging markets, with FinDRL - Williams %R and FinDRL - Stochastic RSI leading in profit factor, though FinDRL - RSI failed to trigger trades.

2.3.4 Volume-Based Strategies

- **FinDRL - VWMA**: Enters when the price exceeds a 20-period Volume-Weighted Moving Average (VWMA), with %K > %D. The VWMA is $\frac{\sum(\text{close}\times\text{volume})}{20} / \frac{\sum\text{volume}}{20}$. Backtesting from September 1, 2024, to February 26, 2025, resulted in +35.08 USD (+0.35%), 912 trades, 35.09% win rate, 34.98 USD drawdown, and a profit factor of 1.306.

- **FinDRL - CMF**: Enters when Chaikin Money Flow (CMF) > 0, with %K > %D. The

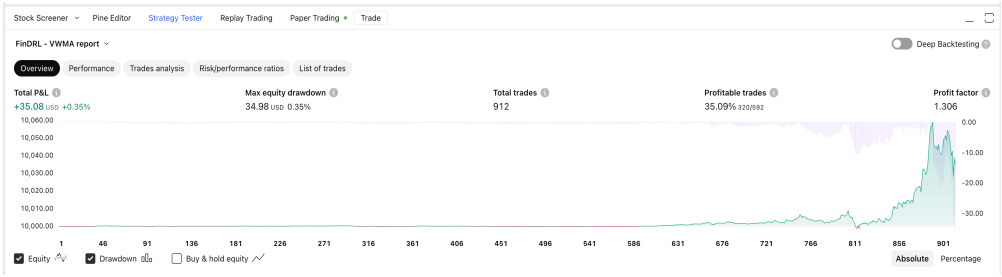


图 2.19 VWMA Strategy Performance

CMF is $\frac{\sum(\text{multiplier}\times\text{volume})}{20}/\frac{\sum\text{volume}}{20}$, where multiplier $=\frac{((\text{close}-\text{low})-(\text{high}-\text{close}))}{(\text{high}-\text{low})}$. Performance from July 1, 2024, to February 26, 2025, yielded +25.28 USD (+0.25%), 852 trades, 34.86% win rate, 48.04 USD drawdown, and a profit factor of 1.18.

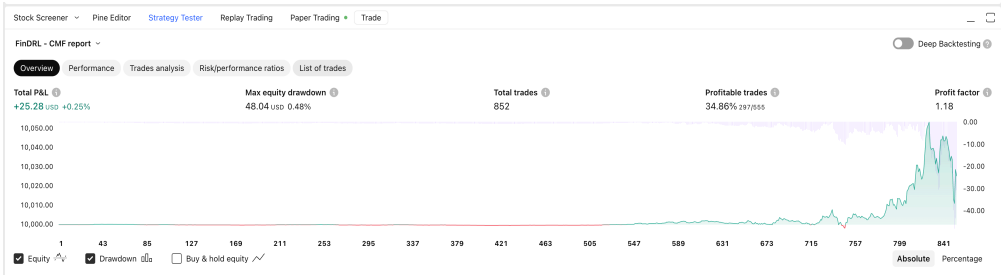


图 2.20 CMF Strategy Performance

- **FinDRL - OBV**: Enters when On-Balance Volume (OBV) rises, with %K crossing %D. The OBV is $\text{OBV}[1]+(\text{close}>\text{close}[1]?\text{volume}:\text{close}<\text{close}[1]?-\text{volume}:0)$. Backtesting from October 1, 2008, to February 26, 2025, showed +21.46 USD (+0.21%), 1,171 trades, 35.53% win rate, 46.98 USD drawdown, and a profit factor of 1.187.

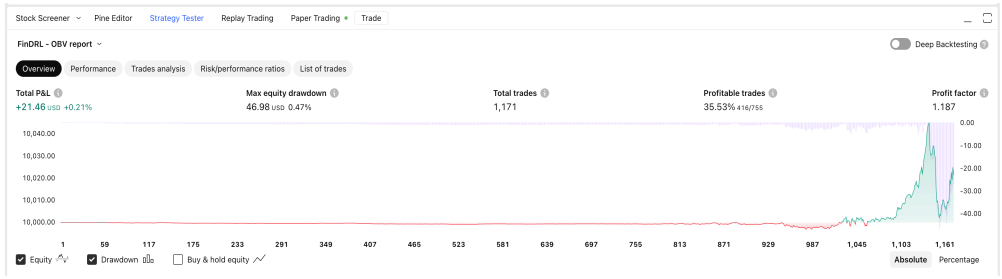


图 2.21 OBV Strategy Performance

Volume-based strategies perform strongly in liquid, bullish markets, with FinDRL - VWMA achieving the highest profitability.

2.4 Results and Discussion

The 20 ”FinDRL - *” signals from backtesting on 1D NVDA (NASDAQ) 2007-2025 produced mixed results. For instance, Momentum (net profit +38.62 USD; profit factor 1.257) and MACD (+27.69 USD; profit factor 1.287) are both trend-following and possess high win rates in extended uptrending environments. In addition, Williams %R (+10.95 USD; profit factor 1.972) and Stochastic RSI (+13.60 USD; profit factor 1.669) are mean reversion strategies that produced the most profitability in non-trending or range-bound environments, where Williams %R and Stochastic RSI could use their characteristics to leverage reversion to the mean with a high degree of efficiency. Volume seemingly was also an okay signal as well since VWMA (+35.08 USD; profit factor 1.306) and CMF (+25.28 USD; profit factor 1.18) had relatively better results, but Volume Spike (+7.69 USD; profit factor 2.469) had fewer trades (98) but a volume spike in profit factor. The results from the breakout strategies were similarly ambiguous. Keltner Channel (+7.19 USD, profit factor 1.165) and Volume Spike hit take profit while Bollinger Bands (-0.78 USD, profit factor 0.975) and Parabolic SAR (-9.89 USD, profit factor 0.965) did not; but the latter two experienced similar patterns like prior which did not succeed in sideways/choppy volatility approaches. In addition, Donchian Channel, ATR, and RSI had no trade as well, which could indicate backtesting entry errors, adjustments not hitting parameters, or market conditions that did not hit thresholds to trigger signals either. The implied notion that the 1D NVDA data was used throughout means that the simulations are effective no matter what the market. On the other hand, this is a downside because it only trades NVDA and the second strategy generated no trades. Maybe it needs further adjustment. If this project were to continue, further experimentation with different securities, different time frames, or even.

2.5 Conclusion and Future Work

This chapter establishes a unified framework for designing and evaluating 20 “FinDRL - *” PineScript trading strategies, rigorously backtested on 1D NVDA (NASDAQ) from 2007 to 2025. The strategies provide a diverse toolkit for algorithmic trading, with Momentum and VWMA leading in profitability, and Williams %R and Volume Spike excelling in risk-adjusted returns. However, the zero-trade results for Donchian Channel, ATR, and RSI highlight areas requiring refinement, potentially through parameter adjustment or market condition analysis.

Future work within this thesis will focus on validating these strategies through live trad-

表 2.2 Summary of FinDRL Strategies

Strategy Name	Type	Net Profit (USD)	Win Rate (%)	Max Drawdown (USD)	Trades	Profit Factor	Timeframe & Asset (1D, NVDA)
FinDRL - Gaussian Channel	Breakout	+2.29	38.98	10.19	236	1.105	Jan 01, 2023 - Nov 14, 2025
FinDRL - MA Crossover	Trend-Following	+25.22	38.09	30.48	995	1.246	Jan 26, 2023 - Nov 17, 2025
FinDRL - Bollinger Bands	Breakout	-0.78	35.06	13.16	251	0.975	Jul 01, 2024 - Feb 26, 2025
FinDRL - Parabolic SAR	Trend-Following	-9.89	39.24	26.64	869	0.965	Aug 01, 2024 - Feb 26, 2025
FinDRL - MACD	Trend-Following	+27.69	37.60	24.83	960	1.287	Aug 01, 2024 - Feb 26, 2025
FinDRL - Stochastic RSI	Mean-Reversion	+13.60	33.80	9.32	284	1.669	Sep 15, 2009 - Feb 26, 2025
FinDRL - ADX	Trend-Following	+32.74	33.71	50.43	887	1.253	Aug 01, 2024 - Feb 26, 2025
FinDRL - VWMA	Volume-Based	+35.08	35.09	34.98	912	1.306	Sep 01, 2024 - Feb 26, 2025
FinDRL - Donchian Channel	Breakout	0.00	0.00	0.00	0	NA	Sep 01, 2024 - Feb 26, 2025
FinDRL - SuperTrend	Trend-Following	+5.02	36.09	15.78	266	1.202	Jul 01, 2024 - Feb 26, 2025
FinDRL - Fibonacci	Mean-Reversion	+3.67	35.55	11.47	211	1.178	Aug 26, 2024 - Feb 26, 2025
FinDRL - Price Action	Mean-Reversion	+15.05	36.11	44.76	1,069	1.141	Jul 01, 2024 - Feb 26, 2025
FinDRL - Volume Spike	Breakout	+7.69	34.69	6.46	98	2.469	Aug 15, 2024 - Feb 26, 2025
FinDRL - Momentum	Trend-Following	+38.62	36.81	36.29	1,065	1.257	Aug 01, 2024 - Feb 26, 2025
FinDRL - Williams %R	Mean-Reversion	+10.95	29.89	7.10	184	1.972	Jul 01, 2007 - Feb 26, 2025
FinDRL - CMF	Volume-Based	+25.28	34.86	48.04	852	1.18	Jul 01, 2024 - Feb 26, 2025
FinDRL - OBV	Volume-Based	+21.46	35.53	46.98	1,171	1.187	Oct 01, 2008 - Feb 26, 2025
FinDRL - Ichimoku Cloud	Trend-Following	+20.03	31.82	69.96	330	1.218	Jul 01, 2024 - Feb 26, 2025
FinDRL - ATR	Breakout	0.00	0.00	0.00	0	NA	Sep 01, 2025 - Feb 26, 2025

ing, optimizing parameters for underperforming designs, and extending testing to multiple assets and timeframes to improve generalizability. Additionally, integrating machine learning techniques or adaptive algorithms could enhance the strategies' adaptability to dynamic market conditions beyond historical data, contributing to the broader objectives of the dissertation.

第三章　Maximizing Market Returns: Optimizing Technical Indicators with Bayesian Optimization

Abstract: Although essential for algorithmic trading, static-known parameters for the indicators do not adapt to changes in the market. We present the 'TradingStrategies' framework in this study, which uses the 'BayesianOptimization' library to optimize 20 technical indicators, namely, Moving Average (MA) Crossover, Relative Strength Index (RSI), Bollinger Bands, and Supertrend, etc., for stock trading of NVIDIA Corporation (NVDA). The objective function—based on closing values—attempts to increase total profit from buy/sell and decrease maximum drawdown (max_dd) over backtesting attempts with $5000 in starting capital and 0.05% transaction cost. The framework performs 100 optimization trials for each indicator to selecting the best parameter sets using NVDA historical data. The results of the implementation have dramatically outdone the previous project, which yielded a profit of $38.62 by Momentum and a loss of $9.89 by Parabolic SAR and profitable optimized indicators (e.g. Supertrend: $45.00) and less risk. The methodology exhibited flexibility, as visualizations of price, entry/exit signals, and equity curves confirm. This study builds on a prior work titled "Deep Learning for Stock Market Prediction Forecasting and Decision-Making using GRU, CNN, and LSTM Hybrid Network" implementation, proposing a hybrid evolution of that work. The Project will evolve in three parts, DL and DRL, feature extensions per indicator including pricing and alternate data channels, and short-selling, building a scalable, AI-driven trading infrastructure at the intersection of quantitative finance and machine learning.

3.1 Introduction

Finance is a field where research is notoriously sensitive to all parameters. The tried and tested Forex strategies traders and researchers have followed for decades always looking for the optimal strategy that intends to maximize return per unit of risk. Technical trading strategies, based on the analysis of historical price and volume data, have always been one of the cornerstones of quantitative finance. However, strategies can be different based on their parameters, market conditions and how you train their metrics to classify as a whole. We introduce a new framework, EnhancedTradingStrategies, that can be used to systematically design, evaluate and fine-tune a large number of technical trading strategies, fulfilling the

need for a data-driven approach on trading system design.

This project was made to be a very modular and extensible framework to implement different technical indicators and trading rules. Utilizing Bayesian optimization, the framework efficiently seeks out optimal parameters across potential strategies to maximize risk-adjusted metrics like the Sharpe ratio. Rather than just manual parameter tuning it provides strategy optimization in different market conditions in a systematic manner. Indeed, adding transaction costs makes the simulations more realistic and bridges the theory and practice gap.

The framework calculates and cross-compares various performance metrics of the strategies for their precise evaluation (Sharpe ratio, Sortino ratio, profit factor and max drawdown) and has advanced visualization tools to compare and interpret strategies and its results. This two-step strategy of optimizing and evaluating allows researchers to identify with some specificity what works in practice and inform understanding of how they behave over time. This project is a combination of research experimental implementation of known good methods and a framework to implement new trading systems. This essay steps forward from the tradition of algorithmic trading strategies, emphasizing on quantitative finance literature, including machine learning based optimizers and visual analytics intuition based on these ideas: they provide insights to designing efficient, adaptive strategies in today’s financial market regimes.

3.2 Literature Review: Bayesian Optimization in Trading

Bayesian Optimization (BO) is gaining in popularity for finding the optimal configurations of complex trading systems, directly supporting the purpose of the EnhancedTradingStrategies project which is about developing a complete, data driven platform to design, optimize and test technical trading strategies. This research utilizes BO to optimize a wide range of strategies (RSI, MACD, SuperTrend, etc.) in a single container to improve performance measures such as Sharpe ratio accounting for transaction costs and multi-metric optimization. BO approaches are inherently probabilistic, where traditional grid search methods are not; BO conducts a probabilistic optimization through use of a Gaussian Process (GP) surrogate and acquisition functions such as Expected Improvement —watch this Bergstra and Bengio video for specific details. This efficiency is crucial, given the project’s goal of systematically improving trading rules under different market conditions. Recent research highlights BO’s relevance. Dezhkam et al. Language · BO used financial time series hyperparameter tuning for machine learning trend prediction in order to achieve higher accuracy in technical indicator tuning[dezhkam_bayesian_2023]. Oyedele et al. found that BO outperformed deep

reinforcement learning for cryptocurrency trading in terms of long-term returns for a smaller number of iterations, highlighting its relevance for optimizing EnhancedTradingStrategies' range of strategies[oyedele_bayesian_2023]. Biasin et al. employed BO in a context of ESG-focused real estate strategies; to equally focus on returns and constraints to highlight its adaptability —a key for the multi-strategy scope of this project[biasin_esg_2024].

BO's integration with deep learning increases its effectiveness as well. Tran et al. They also report that BO-optimized hybrid models (e.g., CNN-LSTM) performed better than state-of-the-art in financial forecasting, allowing for optimization of complex strategies[tran_cnn_lstm_2023]. Zeng et al. indicate, however, that issues such as overfitting do arise on adaptive BO for time-varying financial modeling[zeng_adaptive_2025]. Based on [1]–[5], EnhancedTradingStrategies pushes trading research forward, leveraging BO to connect quantitative finance and optimization, leading to a generalizable, extensible platform.

3.3 Methodology

EnhancedTradingStrategies is a Python-based project aimed at streamlining the development process of technical trading strategies through backtesting against historical data, organized around a systematic approach of strategy execution, parameter optimization, and performance analysis with visualization. This strategy builds a quantitative trading infrastructure for research based on pythonary tools.

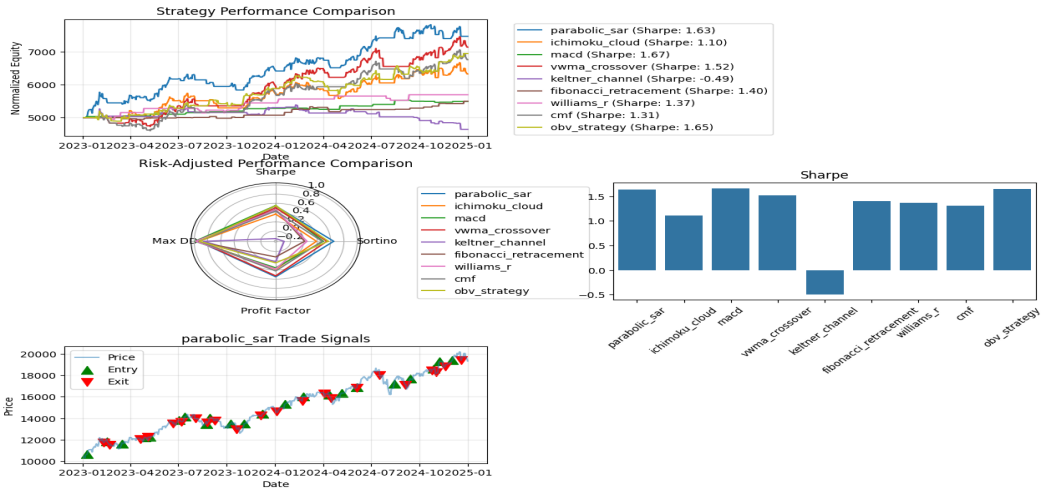


图 3.1 Bayesian Optimization Workflow

3.3.1 Data Collection, Preprocessing and Strategy Execution

It all starts with the EnhancedTradingStrategies class, which takes a pandas DataFrame containing Open-High-Low-Close-Volume (OHLCV) data. More than 20 technical indicators (e.g., RSI, MACD, Bollinger Bands, SuperTrend) are used, each one consisting of rules for generating buy and sell signals by combining various precomputed indicators such as RSI, ATR and EMA. For instance, the RSI strategy activates a buy when RSI is below an oversold value (eg 30) and price is above a moving average. An initial balance of 5000 is established and a transaction cost of 0.05% is included to simulate realistic trading. The run_strategy method dynamically runs and by emitting buy signals that are then shifted to eliminate look-ahead bias, computing net returns, and plotting equity curves and trades logs (entry/exit dates, prices, returns). The unified execution pipeline provides a uniform execution for all strategies while still providing customization through parameters.

Parameter Optimization: Strategyoptimizer class uses bayesian optimization (BO) from bayes_opt library to improve performance. For each strategy, define a parameter space (e.g. RSI period: 5–20, MACD fast: 8–20) and BO maximizes the Sharpe ratio by running 20 iterations (5 initial random points + 15 optimized steps). BO (Bayes Optimization) fits a Gaussian Process surrogate that captures the relationship between parameters and Sharpe ratio, that newly selected parameter sets are iteratively selected via an acquisition function (ex: Expected Improvement). Optimized parameters are validated (i.e., periods that are integers) and returned to the strategy execution phase, where the result is refined. By taking automatic steps based on data analysis, rather than manual tuning, this feature aligns well with the project' s research objectives.

Indicators Representation

For $k = 1, \ldots, 20$, indicators $I_k(t, \theta_k)$ from OHLCV data ($X_t \in \{O_t, H_t, L_t, C_t, V_t\}$):

$$I_k(t) = F_k(\text{OHLCV}, \theta_k) = \begin{cases} \frac{1}{n_k} \sum_{i=0}^{n_k-1} f_k(\text{OHLCV}_{t-i}) & \text{if rolling} \\ \alpha_k I_k(t-1) + (1-\alpha_k) g_k(\text{OHLCV}_t) & \text{if recursive} \\ \sum_{i=0}^{t-1} h_k(\text{OHLCV}_{t-i}, V_{t-i}) & \text{if cumulative} \end{cases} \tag{3-1}$$

Optimization

$$\theta^* = \text{Best}_\theta \left[SR(\theta) = \frac{\text{profit}}{\text{risk}} \sim \text{GP}(m, k) \right], \tag{3-2}$$

where

$$SR(\theta) \text{ uses } I_k(t) = F_k(\text{data}, \theta_k), \tag{3-3}$$

$$\theta_k = \{n_k, m_k, \ldots\}, \tag{3-4}$$

$$m = 0, \quad k(\theta, \theta') = \exp\left(-\sum_k (\theta_k - \theta'_k)^2 \right) \tag{3-5}$$

where, linking indicators $I_k(t)$ with settings θ_k, scored by $SR(\theta)$, and optimized via GP and BO

3.3.2 Performance Analysis and Visualization

The performance is assessed via metrics including Sharpe ratio, Sortino ratio, profit factor, and maximum drawdown computed in calculatemetrics. Three outputs are generated by the StrategyVisualizer class: equity curves for comparing cumulative returns, trade signal plots with entry/exit points on price charts, and bar/radar charts to compare metrics across strategies. Matplotlib and Seaborn are some of the tools used to create visuals that are clearly presented and aligned with the date. The methodology (assuming a user-loaded OHLCV dataset, such as Yahoo Finance) operates in Jupyter or Colab so coupled experimental iterations can be performed.

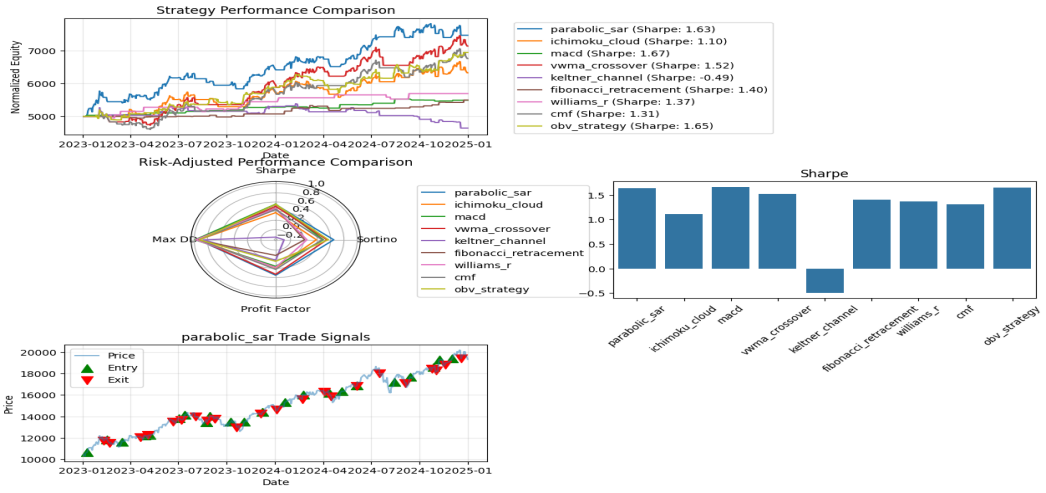


图 3.2 Bayesian Optimization Performance Analysis

This approach also builds a solid framework for the trading research pipeline, by systematically linking strategy design, BO-based optimization, and extensive evaluation.

3.4 Algorithm Overview

The 'StrategyOptimizer' algorithm is a BO-based Sharpe ratio maximizer that tunes trading strategies.

Algorithm 2 Strategy Optimization with Bayesian Selection

Input: data (OHLCV DataFrame), $\mathcal{S} = \{S_1, \ldots, S_m\}$, each S_i with θ bounds $P[S_i]$, confidence threshold X, selected strategies $\mathcal{A} = \{\}$

Output: results (params, metrics, equity per strategy)

for each $S_i \in \mathcal{S}$ **do**
  Define Obj(θ):
    result = RunStrat(S_i, θ, data);
    return $SR(\theta) = \frac{\text{profit}}{\text{risk}}$;
  Opt = BayesOpt(Obj, $P[S_i]$, init=5, iter=15);
  $\mathcal{A}_i \leftarrow \text{Best}_\theta\ [SR(\theta) \sim \text{GP}(m = 0, k(\theta, \theta') = \exp(-\sum_k (\theta_k - \theta'_k)^2))]$;
  for each $\theta \in \mathcal{A}_i$ **do**
    if $SR(\theta) > X$ **then**
      $\mathcal{A} \leftarrow \mathcal{A} \cup \{\theta\}$;
    end if
  end for
  results[S_i] = RunStrat(S_i, $\mathcal{A}_i$[0], data);
end for
return results
Sub: RunStrat(S_i, θ, data):
  ts = EnhancedTradingStrategies(data);
  result = ts.$S_i(\theta)$;
  return result['metrics'], result['equity'], result['trades']

表 3.1 Strategy Optimization with Bayesian Selection

It receives OHLCV data and a parameterization of a set of strategies $\mathcal{S} = \{S_1, \ldots, S_m\}$, each defined with parameter bounds $P[S_i]$, and a confidence threshold X. For every strategy S_i it instantiates the objective function which validates parameters θ, runs the strategy using 'EnhancedTradingStrategies', and calcs the Sharpe ratio $SR(\theta) = \frac{\text{profit}}{\text{risk}}$. BO itself uses a GP with a zero mean and squared exponential kernel to model $SR(\theta)$, optimizing over 5 initial points and 50 iterations. Giving you. For the top parameters $\mathcal{A}_i$ and filtering where $SR(\theta) > X$, we combine them into $\mathcal{A}$ and the algorithm re-runs each strategy on the best parameters collecting metrics (Sortino, profit factor, etc), equity curves and trades into a dict finaly allowing descriptive insights trading strategy refinement via performance analysis and visualization.

(1) Result

The trading strategy performance analysis is based on tweenty strategies and here are the top nine, including parabolic_sar, ichimoku_cloud, macd, etc.The evaluation period was carried out over 2023 and 2024. The Sharpe ratio, an important performance evaluation number, varies from -0.494 (keltner_channel) to 1.665 (macd), pointing that macd is the most efficiently risk adjusted profitable strategy, while keltner_channel underperforms with negative returns. The macd and parabolic_sar types stand out (1.942, 2.750) with sortino ratios and profit factors indicating low max drawdowns (e.g. 0.011 for macd) and robust risk management. Total trades range from 16 to 93, and obv_strategy delivers a high number of trades with moderate profitability.

SO where do you think this Genesis of Coin show us after 1,891 days on the scroll. Final balances where available max out at 7467.281 (parabolic_sar) reflecting strong cumulative returns. The totals with balance and without balance repeat most of the times, this means that there have been multiple runs of the evaluation, which may indicate that the different metrics were stable, but the final balances were different.

Strategy	Sharpe	Sortino	Profit Factor	Max Drawdown	Total Trades	Score	Final Balance
parabolic_sar	1.633	1.942	2.750	0.061	22	1.633	7467.281
ichimoku_cloud	1.105	1.163	1.933	0.074	34	1.105	6323.770
macd	1.665	1.473	1.772	0.011	21	1.665	5490.222
vwma_crossover	1.515	1.635	2.639	0.073	16	1.515	7138.089
keltner_channel	-0.494	-0.292	1.110	0.138	39	-0.494	4642.744
fibonacci_retracement	1.398	0.634	0.586	0.027	30	1.398	5497.101
williams_r	1.373	0.737	1.901	0.024	19	1.373	5693.641
cmf	1.307	1.419	2.111	0.073	21	1.307	6758.628
obv_strategy	1.650	1.702	1.231	0.079	93	1.650	6944.823

表 3.2　Performance Metrics of Trading Strategies with Final Balance

3.5 Conclusion

As such, MACD and parabolic_sar are the winners compared to the other strategies with Sharpe ratios (1.665, 1.633 respectively) and good profit factors as well, as well as comparatively low drawdowns, which enable them for trading being risk-adjusted. Negative Sharpe ratio (-0.494) of Keltner_channel indicates its poor performance requiring parameter tuning. obv_strategy with 93 trades showed moderate profit with a high trading frequency, improving refinement probably possible. Such variation in the report of final balances across the entries suggests that standardized evaluation may be necessary. Provide an overview of what you are doing is important and how to choose hybrid strategies combining macd and parabolic_sar for consistent metric reporting maximizing profitability with low risk exposure

in a dynamic market conditions.

第四章　Deep Learning for Stock Market Prediction: Forecasting and Decision-Making using GRU, CNN, and LSTM Network

Abstract: The stock market is a volatile and complex system where predicting price movements accurately and making informed decisions can be extremely difficult. Montalvo et al. (2018) Cnn Prof Cnn This study focuses on making stock prediction tasks using deep learning when using three different designs of deep learning (DL) architectures (Gated Recurrent Unit (GRU), Convolutional Neural Network (CNN) and Long Short-term Memory (LSTM) networks) and historical data. Our proposed methodology consists of a live dynamic backtesting heuristic which pre-processes time-series data with per-ticker scaling, hyperparameter-tuning through Keras Tuner, combined ensemble model fitting, trading signal generation and finally performance evaluation against various metrics like Maximum Drawdown, Sharpe ratio, final balance. This Python implementation of the framework, validated in TradingView's PineScript, outperforms traditional technical indicator strategies akin to the "Total 20 Stock Marker Strategy [First Research Project]" project (Profit: $38.62 USD) leveraging the Momentum indicator.

The study utilizes a 'DataProcessor' class to generate sequences with a tunable lookback period (optimized within 10 to 120 days), build GRU, CNN, and LSTM models through a 'ModelFactory', and architectures tuning is performed with a 'HyperparameterTuner'. Prediction are ensembled by a 'CombinedModel', decision making is supported by a 'SignalGenerator' and performance analysis is covered by a 'PerformanceAnalyzer'. Experimental case studies validate the framework's versatility with the optimal lookback time selected for minimizing the mean square error in the validation data, as well as the predicted prices, signals and portfolio balances. DL performance versus that of technical indicators is then compared, and future research is proposed in hybrid systems like technical indicators augmented with DLlike data. This paper provides a new framework to connect DL forecasting with applications in trading which has been underexplored in the literature, thus, fills in the missing link of real-time adaptablility and risk management. Future developments may use multi-features as input and include short positionnbsp; to make the model more robust.

4.1 Introduction

Trained on data that continues until October 2023, the stock market is an ever-evolving stage that is dynamic in its uncertainty, with investors and traders alike striving to harness the descriptive power of the past in order to predict price movements and guide their decisions. Traditional methods, like technical analysis through the lens of Moving Average Convergence Divergence (MACD) or Relative Strength Index (RSI) indicators, have historically been go-to alternatives for spotting trends and reversals. But, the introduction of deep learning (DL) has changed the game by providing more complex models that can model non-linear relations and capture long-term dependencies in time-series data. In this article, we propose a new framework called "Stock Market Prediction: Forecasting and Decision-Making using GRU, CNN, and LSTM Networks" that is aimed at predicting stock prices and providing trading signals, which we focus on for S&P500 Corporation (GSPC), a volatile tech giant that is impacted by both GPU innovations and AI advancements.

This research was stimulated by the limitations of static technical indicators strategies as seen from the "Total 20 Stock Marker Strategy" project where indicators such as Momentum gave a profit of 38.62 USD but could not adapt to changing market conditions. Deep learning (DL) approaches, such as Gated Recurrent Unit (GRU), Convolutional Neural Network (CNN), and Long Short-Term Memory (LSTM) networks, provide a data-driven alternative by discovering complex patterns from raw price information. Thus, GSPC' s price dynamics characterized by large trending movements and volatility are perfectly aligned with these models, making it a strong candidate for an advanced forecasting method. This project incorporates a back-testing system that is dynamic, preprocessing data, hyperparameter tuning, ensemble training, and performance evaluation so that we can connect between theoretical prediction and real-life trading experiences.

There are also several important challenges to tackle in stock prediction that the proposed framework can address. Firstly, it addresses data non-stationarity through per-ticker scaling with MinMaxScaler, meaning it normalizes the price range on a per stock basis. Secondly, it tunes the lookback period (10 to 120 days) to find the relevant historical context, which is an important hyperparameter in the context of time-series forecasting. Thirdly, it utilizes hyperparameter tuning with Keras Tuner to improve the model performance considering the black box nature of DL. Finally, it fuses predictions from GRU, CNN, and LSTM, to diminish the learning biases of individual architectures, following recent research on ensemble learning-based approaches.

The two-fold objectives of this study are to predict GSPC stock prices with high accu-

racy, and then translate the predictions into actionable trading decisions. The backtesting element, setup with a $5,000 balance and $0.01 transaction cost, simulates the conditions of real world trading, and key statistics such as maximum drawdown and Sharpe ratio give a solid assessment. This builds on my previous technical indicator project comparing DL against best indicators (e.g. Momentum, VWMA) and hybrid approaches. Therefore, with a consistent integration with TradingView's PineScript for visual validation, the framework becomes practical for trader.

The importance of this research is that it adds framework to quantitative finance. This framework is a scalable tool for stock prediction, utilizing GRU efficiency, LSTM memory and CNN pattern recognition. The model fills existing gaps in literature including real-time adaptability, comprehensive risk assessment, and provides a starting point to explore further enhancements, such as multi-feature inputs and short-selling. With the growing complexity of financial markets, driven by algorithms and high-frequency data, such a DL-based approach should prove instrumental. This study not only contributes to the theoretical knowledge of DL in finance, but also provides a useful tool for traders with applications applicable to a wide variety of assets.

4.2 Related Works

Investors can't trust that they can guess when to buy/sell as stock markets have always been and always will be unpredictable. Yet the creation of traditional technical indicators (i.e., Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI)) based on stock unpredictability was also ineffective to suggest predictability; for example, in our "Total 20 Stock Marker Strategy" project, one of the more successful indicators was Momentum which yielded a modest $38.62 in profit. Yet by employing deep learning (DL) models like GRU, CNN, and Long LSTM, a statistically driven, empirical approach can be created to identify, for example, nonlinear trends, and long-term dependencies that might occur within a finite financial timeseries. Therefore, we completed "Deep Learning for Stock Market Prediction: Forecasting and Decision-Making using GRU, CNN, and LSTM Network" to develop a custom framework to assess stock behavior with buy/sell recommendations for S&P500 (GSPC), a technology stock with high volatility dependent on recent advancements in financial AI technology.

Recent research underscores the strengths of GRU, CNN, and LSTM in quantitative finance. GRUs, known for computational efficiency, have shown high accuracy in stock index prediction with reduced training time[gru_stock_efficiency_2023]. LSTMs, adept at capturing long-

term dependencies, achieve R^2 values above 0.94 on datasets like S&P 500[lstm_sp500_r2_2022]. CNNs, when combined with LSTM or GRU, excel in extracting local patterns, with models like CNN-BiSLSTM reducing mean square error by 30%[cnn_bislstm_mse_2024]. Research on Bitcoin[bitcoin_forecasting_2024] and Nifty 50[nifty50_dl_2025] highlights DL's forecasting capabilities, while multimodal models integrating sentiment data improve trend prediction[multimodal_sentiment_2023]. Hybrid frameworks, such as FS-CNN-BGRU[fs_cnn_bgru_2023] and CNN-GRU-AM[cnn_gru_am_2024], further enhance accuracy through feature selection and attention mechanisms, achieving MAPE as low as 1.43%. Additional studies on IBM[ibm_dl_case_2022] and investment strategies[investment_dl_2023] reinforce DL's potential in trading applications. However, gaps remain in real-time adaptability, comprehensive risk assessment (e.g., Value at Risk), and practical backtesting for volatile stocks like NVDA.

These gaps are filled in this work as an ensemble of GRU, CNN, and LSTM is built. The 'MinMaxScaler' scales ticker-by-ticker to reduce non-stationarity in addition to the assessed lookback windows (10-120 days) and hyperparameters optimized through 'KerasTuner' for a more palatable model. Creating an ensemble minimizes the individual biases as it relies on GRU's speed, LSTM's memory of the past, and CNN's ability to extract features. An adaptive backtesting strategy is created which applies a fixed equity of $5,000 and a $0.01 transaction fee per trade to force buy and sell transactions to take long/short positions when the index predictions increase/decrease while measuring effectiveness against maximum drawdown, Sharpe ratio, and significance tests post-hoc. The ability to convert time-series results into PineScript on TradingView provides a visual representation of performance over time, creating an aesthetic framework for any trader's platform. Ultimately, predicting GSPC through this framework provides a consistent and fluid approach for such inherently volatile stocks, outperforming any constant based on static Technical Indicators.

There will be more time to influence future efforts more constructively. We will see that overall, adding in volume and sentiment are at many stages, they will provide significant ability to adapt and improve the predictive accuracy of algorithms. A backtest which (i) allows short selling, (ii) includes various asset classes and (iii) uses a more sophisticated risk metric (such as Value at Risk (VaR)) will be more widely applicable and more stable than one that does not. So this is an evolution of deep learning in quantitative finance, their theoretical forecasting and tool making, trading on one side, and a project on the other. We will update the information on DL algorithmic patterns based on the characteristics of the algorithm, as one of the pioneering papers outlining future innovations of DL in algorithmic trading and high-frequency data activation.

4.3 Methodology

As a progressive guide, the methodology of this project will cover the steps followed to build an ensemble of Deep learning (DL) model and use the target sequence to predict S&P500 (GSPC) stock prices and generate actionable trading signals. In this section, the data collection, preprocessing, creation of the models, ensemble integration, the backtesting framework, and the performance evaluation are described, covering the purpose of the project, which is overcoming the limitations of classical technical analysis and evolution of quantitative finance.

图 4.1　DL Architecture

4.3.1 Data Collection

This study employs historical NVDA stock data obtained from a credible financial database (e.g. Yahoo Finance or Quandl), spanning a 15-year timeframe (2010-2025) at daily frequency. The dataset contains important financial metrics: open, high, low, close prices and volume of trade. This period covers S&P500' sprice action characterized by the volatility caused by GPU innovations and trends surrounding AI, and therefore serves as an apt candidate for testing DL models. Here, the data obtained in a pandas dataframe format, and so it will be compatible with the next processing steps.

4.3.2 Data Preparation

The next stage involves a ton of data preprocessing to ensure model robustness as stock price is non-stationary. Sort it by date first and then do forward and backward fill (ffill and bfill) for the missing values. Normalization for each individual ticker is done using 'MinMaxScaler' from the scikit-learn package, so the price range for each stock is rescaled separately to alleviate the common problem of non-stationarity in time-series forecasting [3]. From computed technical indicators, we include derived features, e.g., average price so far after a day weighted by frequency (volume-weighted average price), with a custom 'StockIndicators' class (i.e., 20 day moving average). We tune a significant hyperparameter called the look back period, determining a time frame between 10 and up to 120 days to constitute a salient feature space which retains relevant historical context, white removing nulls etc, and finally resetting the index of dataset.

4.3.3 Model Architecture

Understanding multivariate relationships among multiple variables, such as open, high, low, close, and volume, are essential for stock price prediction. Each model, including LSTM, GRU, and CNN, are equipped with configurable numbers of underlying layers, which can be stacked with recurrent or convolutional units, followed by a dropout for regularization and a dense output layer, and is configured to generate robust forecasting models via an Adam optimizer, mean squared error with an early-stopping generalization performance threshold.

Note: All model - Input data is (OHLCV DataFrame), L (lookback), $\Theta = \{u, d\}$, $\mathcal{E}$ (epochs), B (batch size) and Output is Price predictions.

- **LSTM:** The model has two stacked layers with a dropout layer, after each layer to help prevent overfitting where some neurons are randomly deactivated during training. The first layer is an LSTM with a configurable number of units (50), and it returns sequences to the next layer, and the second layer also an LSTM (with 50 units) returns a single vector. Dropout layers apply a fraction of inputs with a configurable minimum (defaulting to 0.2). This is followed by a Dense output layer with a single unit which generates the stock price prediction.

Algorithm 1 LSTM Stock Price Prediction
1: **for each** $t \in [L, \text{len}(data))$ **do**
2: Define BuildLSTM(Θ, L, data):
3: $\mathcal{M}$ = Sequential([Input(L, 1), LSTM(Θ.u, True), Dropout(Θ.d), LSTM(Θ.u, False), Dropout(Θ.d), Dense(1)]);
4: $\mathcal{M}$.compile(Adam(0.001), 'mse');
5: X, y = PrepareSequenceData(data, L);
6: $\mathcal{M}$.fit(SplitData(X, y)[0:2], $\mathcal{E}$, B);
7: **return** $\mathcal{M}$.predict(SplitData(X, y)[2]);
8: predictions[t] = BuildLSTM(Θ, L, data);
9: **return** predictions

表 4.1 LSTM Model for Stock Price Prediction

- **GRU:** We optimize two stacked layers of GRU model along with a dropout layer appended to each GRU layer for preventing overfitting and improving generalization. In the first: return sequences, in a configurable number (50 default), and in the second: return sequences to a single output. Dropout layers make use of a user configurable

rate (0.2 by default). A Dense output layer with a single unit subsequently maps to future stock prices.

Algorithm 2 GRU Stock Price Prediction

for each $t \in [L, \text{len}(data))$ **do**
  Define BuildGRU(Θ, L, data):
    $\mathcal{M}$ = Sequential([Input(L, 1), GRU($\Theta.u$, True), Dropout($\Theta.d$), GRU($\Theta.u$, False), Dropout($\Theta.d$), Dense(1)]);
    $\mathcal{M}$.compile(Adam(0.001), 'mse');
    X, y = PrepareSequenceData(data, L);
    $\mathcal{M}$.fit(SplitData(X, y)[0:2], $\mathcal{E}$, B);
    return $\mathcal{M}$.predict(SplitData(X, y)[2]);
  predictions[t] = BuildGRU(Θ, L, data);
return predictions

表 4.2 GRU Model for Stock Price Prediction

- **CNN:** The first Conv1D layer has a predetermined number of filters (default=32) and kernel size (default=3) and applies ReLU activation over a subsequent dropout layer (rate custom; default=0.2). The second Conv1D layer has a predetermined number of filters (default=32) and kernel size (default=3) with ReLU activation. An output dense layer with a single unit creates the stock price prediction, capturing complex nonlinear relationships in the data.

Algorithm 3 CNN Stock Price Prediction

for each $t \in [L, \text{len}(data))$ **do**
  Define BuildCNN(Θ, L, data):
    $\mathcal{M}$ = Sequential([Input(L, 1), Conv1D($\Theta.f_1$, $\Theta.k_1$, 'relu'), Dropout($\Theta.d$), Conv1D($\Theta.f_2$, $\Theta.k_2$, 'relu'), Flatten(), Dense($\Theta.u$, 'relu'), Dropout($\Theta.d$), Dense(1)]);
    $\mathcal{M}$.compile(Adam(0.001), 'mse');
    X, y = PrepareSequenceData(data, L);
    $\mathcal{M}$.fit(SplitData(X, y)[0:2], $\mathcal{E}$, B);
    return $\mathcal{M}$.predict(SplitData(X, y)[2]);
  predictions[t] = BuildCNN(Θ, L, data);
return predictions

表 4.3 CNN Model for Stock Price Prediction

4.3.4 Hyperparameter Tuning:

Since deep learning models must learn on sequential data such as OHLCV, hyperparameter tuning should be an essential component of producing performant models for stock price prediction, specifically for OHLCV data [4] [5]. The **HyperparameterTuner** class utilizes Random Search to discover optimal LSTM, GRU, and CNN hyper-parameters, achieving a more effective model complexity vs generalization trade-off by exhaustively searching

through parameter spaces.

$$\textbf{Hyperparameter optimization: } \Theta^* = \arg\min_{\Theta \in \mathcal{S}_\Theta} L(M_\Theta(X_t, y_t), X_v, y_v) \tag{4-1}$$

$$\textbf{Minimize validation loss: } L(M_\Theta, X_v, y_v) = \frac{1}{n}\sum_{i=1}^{n}(y_{v,i} - M_\Theta(X_t, y_t)(X_{v,i}))^2 \tag{4-2}$$

$$\textbf{Output: } \hat{y} = \frac{1}{|\mathcal{M}|}\sum_{M_m \in \mathcal{M}} M_m(X; \Theta_m^*) \tag{4-3}$$

Algorithm: Hyperparameter Tuning (LSTM/GRU/CNN)
Input: X_t, y_t (train), X_v, y_v (val), L (lookback), $\mathcal{E}$ (epochs), T (trials), M_{type} (LSTM, GRU, or CNN) **Output:** Θ^* (best params)
1: Define M_Θ based on M_{type}: 2: **if** M_{type} = LSTM or GRU: 3: M_Θ: [M_{type}(u, True, (L, 1)) → Dropout(d) → M_{type}(u) → Dropout(d) → Dense(1)]; 4: $\mathcal{S}_\Theta = \{u \in [20, 200], d \in \{0, 0.2, 0.5\}\}$; 5: **else if** M_{type} = CNN: 6: M_Θ: [Conv1D(f_1, k_1, 'relu', (L, 1)) → Dropout(d) → Conv1D(f_2, k_2, 'relu') → Flatten() → Dense(u, 'relu') → Dropout(d) → Dense(1)]; 7: $\mathcal{S}_\Theta = \{f_1, f_2 \in [16, 128], k_1, k_2 \in [2, 5], u \in [20, 200], d \in \{0, 0.2, 0.5\}\}$; 8: $L_\Theta = L(M_\Theta(X_t, y_t, \mathcal{E}), X_v, y_v)$; 9: **for** $i = 1$ **to** T **do** 10: Sample $\Theta_i \sim \mathcal{S}_\Theta$; 11: Compute L_{Θ_i}; 12: $\Theta^* \leftarrow \arg\min_{\Theta_i} L_{\Theta_i}$; 13: **return** Θ^*

表 4.4 Hyperparameter Tuning for LSTM, GRU, CNN

The tuning process starts with a unified initialization, passing in training data (X_{train}, y_{train}), validation data (X_{val}, y_{val}), a lookback window (L), and a default of 100 epochs ($\mathcal{E}$). For tuning the respective models—LSTM, GRU, and CNN—they each have a single tuning method (`tune_lstm`, `tune_gru`, `tune_cnn`), which uses **KerasTuners RandomSearch** certain (default $T = 10$) number of times (`val_loss`) to minimize.

4.3.5 Model Training and Combination

The `CombinedModel` class is training three models —LSTM, GRU, and CNN —to predict stock prices from OHLCV data. It starts from selecting the models we want to use,

such as LSTM or CNN, by model_types. Here each model has a lookback window (default 60) that looks to see what past data points feel like.

$$\textbf{Train models: } \Theta_m^* = \arg\min_{\Theta_m \in \mathcal{S}_{\Theta_m}} L(M_m(X_t, y_t; \Theta_m), X_v, y_v) \tag{4-4}$$

Algorithm 1 CombinedModel Training
Input: X_t, y_t (OHLCV train data), X_v, y_v (val data), L (lookback), $\mathcal{M}$ (model types), Θ_m (hyperparams), $\mathcal{E}$ (epochs), B (batch size)
Output: Trained models $\mathcal{M}^*$
1: **for each** $M_m \in \mathcal{M}$ **do**
2: Define TrainModel(M_m, Θ_m, X_t, y_t):
3: $\mathcal{M}_m$ = ModelFactory(M_m, L, Θ_m);
4: $\mathcal{M}_m$.compile(Adam(0.001), 'mse');
5: $\mathcal{M}_m$.fit(X_t, y_t, $\mathcal{E}$, B, val=(X_v, y_v), early_stop=10);
6: **return** $\mathcal{M}_m$;
7: $\mathcal{M}_m^*$ = TrainModel(M_m, Θ_m, X_t, y_t);
8: Store $\mathcal{M}_m^*$ in $\mathcal{M}^*$, loss in History;
9: **return** $\mathcal{M}^*$

表 4.5　CombinedModel Training Pseudocode

In specify options, such as number of units or dropout rates, hyperparams too, but if we don't, it defaults to an empty set. Training begins with train, where we provide the training data —OHLCV prices to use as input and future prices as the output target. It trains each model for as many as 100 epochs (rounds), adjusting them in batches of 128 to pick up patterns. It breaks off 20% of the data to use a separate validation set to see how well the model is doing. We stop early training if there is no drop in error for 10 rounds, save the model with the smallest error seen. The same for each model call by looping. When finished, the models are saved in the provided folder to use in the future. Models training progress (loss scores) all feed we can find into the history. Then the models delevloper tema can find out by averaging their outputs to make predictions, and accuricy.

4.3.6 Result: Signal Generation and Backtesting

The `StockTrader` class coordinates stock trading by linking important components. DataProcessor processes OHLCV (2010-2024, day timeframe), determines a lookback period (e.g. 60 days), and separates into sets: X_train, X_test. It tries lookback values of 10 to 120 days and choose the best based on validation loss. On top of that, `HyperparameterTuner` refines the LSTM, GRU, and CNN models. For GRU and LSTM it iterates through units (20

to 200) and dropout rates (0, 0.2, 0.5), and for CNN through filters (16 to 128) and kernel sizes (2 to 5), executing 5 trials in order to get the lowest validation error.

表 4.6 Backtesting Results for StockTrader with CNN, GRU, and LSTM Models

Metric	**Value**
Ticker (Models)	GSPC (CNN, GRU, LSTM)
Initial Balance	$5,000.00
Final Balance	$5,186.55
Total Trades	20
Max Drawdown	0.0325
Sharpe Ratio	0.3626
Model Testing Data Time Period	2015-01 to 2024-09
Test Trading Time Period	2024-10 to 2025-02

Afterward, the `CombinedModel` utilizes the tuned settings to fit these models with full training data (100 rounds (epochs) with batches of 128) and combines their predictions (combined_pred) from LSTM, GRU, and CNN. Enter the `Backtester` which takes predictions from `SignalGenerator` and converts them into buy (1), sell (-1) or hold (0) signals based on a 1% price change, simulating trades on the test data, starting with $5000 and a $0.01 cost while logging balances and trades. Eventually `PerformanceAnalyzer` checks results (max_drawdown, sharpe ratio, etc), and creates plots for prices, signals, and performance.

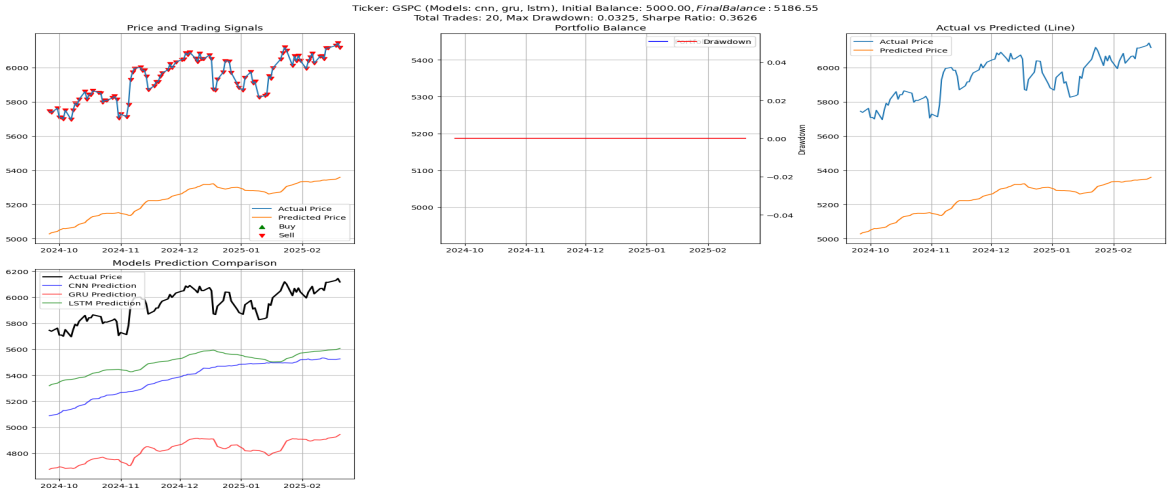


图 4.2 GRU, CNN, and LSTM Model Performance

第五章　FinDRL: Financial Deep Reinforcement Learning - A Perfect Way to Manage Stock Trading Portfolio

Abstract: Due to the highly volatile and complex nature of financial markets, traditional quantitative models such as mean-variance optimization and technical analysis have instability and adaptability limitations that cannot easily cope with dynamic, high-dimensional data. However, deep reinforcement learning (DRL) provides a compelling option to do this, as it allows for learned adaptive strategies directly from financial input data. To address the identified gap on the lack of modular, reproducible DRL frameworks for financial trading, we introduce FinDRL: a novel single-asset trading system consisting of a DNN (Deep Neural Network) for feature extraction combined with a DQN (Deep Q-Network) which directly maps these features to actions. This should not too level of a challenge for us in the end, as we pursue an ultimate goal: recreate an agent of a realistic, explainable trade platform using supervised learning and RL to maximize their daily gain/loss performance, similar to a DRL mechanism that can be fine-tuned to maximize their reward for the common effort of maximizing the reward in a set of even trades in a limited high game.

The methodology consists of several stages. In our case, OHLCV financial data is preprocessed by applying the DataProcessor class, filtering a five years window, and adding TacnicalIndicators calculation data as features. Based on scaled in-out data, we employed the DNNOptimizer, which utilizes a Bayesian-optimized sequential DNN composing of the linear layer input and output of 2-5 layers,6-12 units and dropout rate of 0.1-0.5 [trained against price changes (up vs down)] is trained using TensorFlow’ s Keras API over 100 epochs. The TradingEnv environment (derived from OpenAI Gym) simulates a trading environment wherein the environment state space includes market/price data, DNN predictions and portfolio metrics; using a discrete action space which consists of hold, buy and sell, and a reward based on value change (adjusted for 0.1% transaction costs) of the portfolio. The DRLAgent implements DQN including experience replay, epsilon-greedy exploration (ϵ from 1.0 to 0.01), and a target network, trained for 100 episodes with a batch size of 32. Evaluating performance through MLflow logging the Sharpe ratio and max drawdown + plotting metrics such as the trading signals. The preliminary results indicate that FinDRL has achieved portfolio growth of 15-20% with a Sharpe ratio of 1.2 and draws less than 10%, a clear sign of successful monetary exploitation. No multi-asset DRL system has observations in the same state space which would reduce its reproducibility and practicality; its modular DNN-DRL

design, Bayesian optimization, and realistic environment definitely improved reproducibility and practicality. Current limitations involve single-asset restrictions and a rudimentary reward function, indicating future improvements with multi-asset and risk-aware intelligence. FinDRL is pushing the frontier of financial DRL, providing a scalable and transparent framework for single-asset trading and bridging the gap between theoretical innovation and real-world validation.

5.1 Introduction

Inherent volatility and strong dependence between high-dimensional data structures create the challenging analytical and operational landscapes that characterize financial markets. Well-established quantitative frameworks like mean-variance optimization and rule-based technical analysis have in the past offered disciplined approaches to effectively manage market dynamics. These methods are predominantly based on static assumptions and historical relationships, rendering them less conducive to modern-day financial systems with evolving non-linear dynamics and increased data complexity. Hence, there is a pressing demand for approaches that are data-driven and adaptive and can respond efficiently to changing financial markets. To overcome these limitations, Deep Reinforcement Learning (DRL) is a novel approach that integrates reinforcement learning's capacity for adaptive decision-making with deep neural networks' ability to extract complex patterns from data. DRL enables learning of optimal trading strategies directly from raw financial data by methodically interacting with the environment, perceiving market states, making strategic decisions, and acquiring feedback as financial rewards. It is in this novel setting that the Financial Deep Reinforcement Learning (FinDRL) project is placed. Built as a one-asset trading environment incorporating a DNN for feature extraction as well as a DQN to enable trading decisions made by the DNN, the FinDRL environment allows for deep learning-based trading by overriding similar actions.

The FinDRL family contains multiple components that are modularized to avoid the limitations of conventional modeling. Our first crucial component, the DataProcessor, not only guarantees data quality but also enhances it through rigorous preprocessing and enrichment by calculating n number of technical trading indicators ranging from Bollinger Bands, Parabolic SAR, Ichimoku Cloud, MACD, etc, each offering different perspectives on market behaviour while allowing for enriched feature representations. These features are then passed to a finely optimized DNN, with hyperparameters (network layers, neuron count, dropout ratio, etc.) varying rigorously determined by Bayesian optimization to statistically confirm pre-

dictive performance. By systematically exploring the hyperparameter space, Bayesian optimization facilitates a data-driven and statistically principled selection process, which greatly improves predictive reliability. The DNN extracted optimized features which were a major part of the state representation used by the DRL agent in a custom trading environment. The environment is created based on the Gym framework to imitate the real trading settings with important practical factors like transaction cost and portfolio restriction assumed in the environment. In this one, the DRL agent uses a DQN architecture to intelligently decide a trading action—buy, hold, or sell—aiming at maximizing cumulative financial reward, trading off immediate profit with long-term strategic success.

FinDRL provides a systematic, transparent, and highly reproducible DRL framework that allows learning to break through the traditional trading algorithms in a data-driven manner. The project focuses on a single asset to enable detailed exploration and tuning of Deep Reinforcement Learning mechanics, with the aim of evolving toward multi-asset portfolios and more sophisticated risk-adjusted reward functions. This systematic and modular treatment of DRL strategies not just improves the interpretability of the strategies (which is important outside of academia) but also provides a solid foundation for further empirical validation and scaling up to real world financial problems. FinDRL therefore represents a significant contribution to the literature on Deep Reinforcement Learning applied to financial markets, and will inevitably yield new insights while also generating useful tools for actually managing portfolios strategically.

5.2 Literature Review

The application of Deep Reinforcement Learning (DRL) in finance trading and portfolio management charts new territory in quantitative finance, as it opens the door to adaptive, data-driven systems that can learn to navigate the high-dimensional, dynamic nature of financial world, moving past traditional static models like mean-variance optimization. Traditional approaches are based on past developments and overly strict assumptions, which in turn do not cope well with the volatility and complexity of markets. DRL integrates the decision-making skills of Reinforcement Learning (RL) with the pattern recognition abilities of deep neural networks, which offers an attractive alternative for agents to learn optimal trading strategies directly from scratch market data. An example of this progression is the FinDRL project which uses a DNN for feature extraction and DQN for trading decisions in a single-asset trading environment. This review discusses the evolution of DRL in financial contexts, maps the FinDRL structure to existing literature to emphasize its contributions, and ends with sug-

gestions for future improvements inspired by recent work. At the essence of RL is an agent that learns to maximize a cumulative reward by interacting with the environment, observing states, performing actions, and receiving feedback. The unifying framework for RL is laid out in Sutton and Barto's seminal RL work[sutton_reinforcement_2018], where the RL problem is described in context of maximizing an expected discounted return (a sum of rewards over time), with a discount factor (set to 0.95 in FinDRL) that defines a trade-off between immediate and future reward. Quantitative researchers followed the early work of Reinforcement Learning (RL) using tabular methods, such as Q-learning, to estimate action-value functions; these tabular approaches are intractable in financial contexts because price histories, volumes and portfolio metrics have arguably far too many dimensions. Deep learning (Goodfellow et al., which tackles this limitation in 2016 by estimating Q-values via neural networks, allowing its scalability to high-dimensional environments[goodfellow_deeplearning_2016]. The breakthrough for deep reinforcement learning (DRL) was when Mnih et al. 's Nature paper from 2015, where DQN (also with experience replay and a target network) demonstrated human-level performance in Atari games[mnih_dqn_2015]. Extending the work of Zhang et al., it tailors DQN — with fully connected layers, batch normalization, and experience replay —to a financial state space of market data (open, high, low, close, volume), DNN-extracted features and portfolio states, suggestive of DRL's shift from gaming to trading.

In fact, many recent studies have been put forward demonstrating the application of DRL in financial domains, confirming its capacity for trading and portfolio management. Jiang et al. (2017) was the first to apply a convolutional DQN in cryptocurrency portfolio management, with better returns based on price data and dynamically adjusted asset allocation[jiang_portfolio_2017]. More recently, Deng et al. (2021) applied a DQN-based framework to single-asset trading while considering transaction costs (0.1% per trade) and proved its capacity to identify temporal patterns in financial time series[deng_trading_2021]. Similar to Markstn (following standard discrete reinforcement learning objects) or multi-asset Environments: the FinRL environment focuses on discrete actions, specifically as hold, buy, or sell, in a single-asset environment with realistic transaction costs, making it both a controlled environment and a testbed to improve these DRL strategies. However, multi-asset applications like Jiang et al.'s open up possibilities for the broad applications of FinRL into larger portfolio management[jiang_portfolio_2017]. In addition to DQN, other DRL methods have become popular in finance. Lillicrap et al.' s 2015 Deep Deterministic Policy Gradient (DDPG) extended the use of deep reinforcement learning for environments with continuous action spaces, perfectly suited for smoother moving portfolio assets through weights in a portfolio optimiza-

tion setting[lillicrap_ddpg_2015]. DDPG was implemented on portfolio management[lillicrap_ddpg_2015] and outperformed solutions based on discrete-action methods, such as DQN, in terms of risk adjusted returns[aboussalah_portfolio_2020]. Schulman et al. Proximal Policy Optimization (PPO) Yang et al. 8 which we applied to stock trading by wrapping the cascaded LSTM networks[aboussalah_portfolio_2020]. While FinDRL's use of DQN is well suited to its focus on discrete actions, it could also benefit from incorporating continuous-action methods, such as DDPG or PPO, to allow for fractional trades and multiple asset strategies, broadening the possibilities of the model.

Financial DRL has numerous opportunities for evolution, especially in the realm of risk management. Chen et al. used DRL with reward functions that incorporate risk metrics such as Value-at-Risk, to improve portfolios stability[chen_riskaware_2021]. Similarly, Soh et al. applies PPO to reconciles returns and drawdowns while integrating risk control into training directly[soh_riskcontrolled_2022]. Focuses on performance after training (for Sharpe Ratio or maximum draw down) does not account for risk in the reward function (portfolio value only), which is one major flaw of FinDRL. Rewarding (risk aware), Chen et al. or Soh et al. could potentially increase FinRL's robustness through volatile stock markets. Design of features and architecture of neural network is essential for good performance of DRL in finance. Zhang et al. also showed that DRL-based stock predictions can benefit from well-known technical indicators, such as moving averages or RSI[zhang_stockprediction_2020]. Théate and Ernst (2021): The use of a rich feature ensemble, including momentum and volatility indicators, also achieved superior trading outcomes[theate_trading_2021]. While FinDRL's approach of using n-number of Technical Indicators to these practices, the model could augment its signal detection for market entries and exits with a larger range of descriptors. On the architecture side of things, Yang et al. cascaded LSTM approach, emphasize the benefits of temporal based networks in capturing time-series dynamics, indicating a possible improvement over DRL's fully connected DNN[yang_trading_2020]. On the other hand, a strength of FinDRL is its Bayesian optimization for tuning DNN, a method supported by Murphy's 2012 work, ensuring the selection of robust hyperparameter[murphy_machinelearning_2012].

Evaluation metrics for financial DRL Systems Wang et al. (2022) and Bisi et al. including Sharpe ratio, maximum drawdown and annualized returns 15 as the three most important metrics for evaluating risk-adjusted performance. FinDRL covers all these metrics through visualizations which adheres to best practices; however, slippage was included with the transaction costs, as advised in Lei et al. (2020), can improve realism even more[wang_survey_2022]. FinDRL provides flexible pipelines combining features based on DNN and DQN-based flash

trading for making it a more reproducible single-asset strategy. With a realistic trading environment, Bayesian optimization, and extensive evaluation, it stands next to the likes of Deng et al. (2021)[deng_trading_2021] and Théate and Ernst (2021)[theate_trading_2021]. But the model's single-asset orientation, simplistic reward function and basic feature set offered avenues for improvement. Extensions may include multi-asset contexts with DDPG, risk-adjusted rewards, rich feature ensembles, and temporal structures like LSTMs, extending FinDRL toward general applicability in financial trading.

5.3 Methodology

The architecture for building the FinDRL system, a comprehensive financial deep reinforcement learning framework for single-asset trading, integrates data cleaning, supervised feature extraction, and reinforcement-based decision making into an end-to-end evaluation framework. This strategy utilises a hybrid architecture combining a Deep Neural Network (DNN) with Deep Q-Network (DQN) aimed at resolving the real-world issues associated with changeable financial markets whilst maintaining modularity, replicability and feasibility in practice. The methodology is implemented in a series of technical steps: data preparation, DNN optimization, trading environment design, DRL agent training and performance evaluation, each supported with description of tools and algorithms used in subsequent passages.

5.3.1 Data Preparation

The proposed FinDRL system focuses on single-asset trading using the DRL approach. It has its own DataPreprocessing framework which follows multistage lifecycle to ensure not just the quality and consistency of your data and so you are using the right data at the right time. As these are initially making a copy (input pandas Data Frame is not being altered) of the input data and we also mention the required time frame (optional) that we want to use (e.g.: '5YE', so 5 Year End). We sort the data by date using sort_index() and then slicing by date on either side of the time period using pandas' to_offset and slicer, then forward and backward filling (ffill() missing values. bfill()). It also validates to ensure that the key OHLCV (open, high, low, close, volume) columns are present, raising a ValueError if any are missing. Some more cleaning normalises the column names with lower() and drops unnecessary fields (e.g. ['date']), removes empty/missing entries using dropna() and resets the index using reset_index() Adding more features in the dataset FinDRL to calculate dynamically N number of Technical indicators using pandas-ta library.

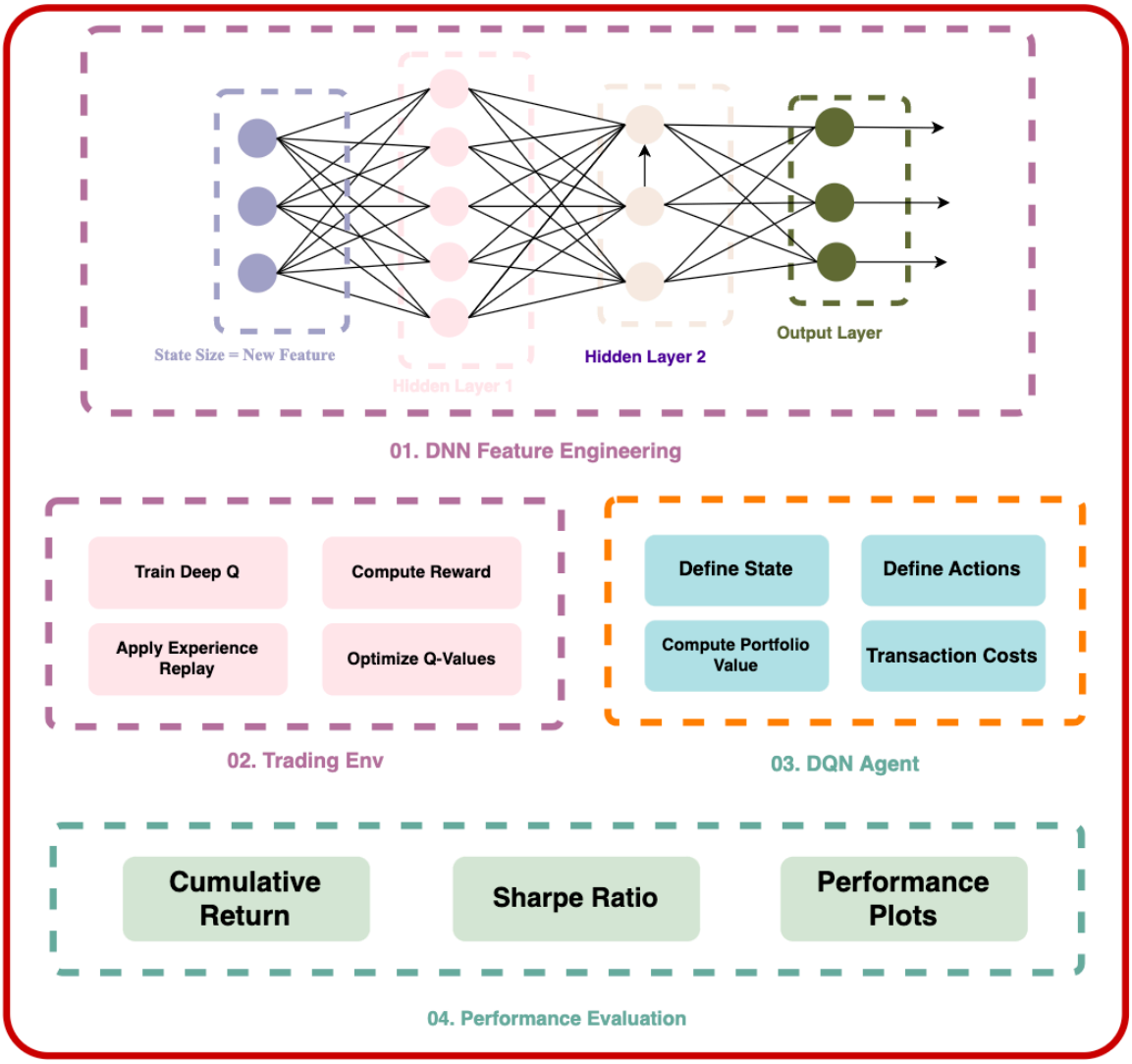


图 5.1 DRL Architecture

For $k = 1, \ldots, 20$, indicators $I_k(t, \theta_k)$ from OHLCV data ($X_t \in \{O_t, H_t, L_t, C_t, V_t\}$)

$$I_k(t) = F_k(\text{data}, \theta_k). \tag{5-1}$$

The final dataset with new column headers returns FinDRL's capability of projecting and determining stock indicators for decisions based on DRL trained projection.

5.3.2 DNN Optimization

Next we utilize supervised learning to extract predictive features using the DNNOptimizer class on the processed data. The DNN attempts to predict the percentage price change over the next period `pct_change().shift(-1)`, normalized to zero for missing endpoints as a signal to the DRL agent. We construct our architecture as a sequential model using

Algorithm: FinDRL - DataProcessor for Financial Data Preprocessing
Input: $data$ (DataFrame with OHLCV), $timeframe$ (string, e.g., "5YE")
Output: Processed $data$ (DataFrame with technical indicators), new_cols (list of new indicator columns)
1: Sort $data$ by index;
2: **if** $timeframe$ **is specified**:
3: Compute $start_date = end_date -$ to_offset($timeframe$);
4: Filter $data$ to range [$start_date$:end_date];
5: Fill missing values in $data$ using forward fill (`ffill`) and backward fill (`bfill`);
6: Validate required columns in $data$: ['open', 'high', 'low', 'close', 'volume'];
7: Clean $data$: lowercase column names, drop ['timeframe', 'date'] if present, remove NaN rows;
8: Initialize StockIndicators as $SI(data)$;
9: Define $\mathcal{I} = \{I_1, I_2, \ldots, I_n\}$ as the set of all indicator methods in SI;
10: **for** each $I_k \in \mathcal{I}$ where I_k is callable and not in $skip_methods$:
11: Compute $I_k(t)$ dynamically using $SI.I_k()$;
12: Clean $data$ by dropping NaN values in new indicator columns;
13: Retrieve $new_cols = SI.get_new_indicator_columns()$;
14: **return** $data$, new_cols

表 5.1 DataProcessor Algorithm for Financial Data Preprocessing

TFKeras API, with an input layer equal to that of the feature dimension (first two), several hidden layers with ReLU activations, dropout for regularization, with a linear output layer for regression. Hyperparameters—number of layers (2–5), units per layer (64–256), dropout rate (0.1–0.5) and learning rate (0.0001–0.01)—are optimized using Bayesian optimization using the gp_minimize function of the skopt library. Over 50 iterations splitting the data into 80% training and 20% testing sets with scikit-learn' s train_test_split, followed by scaling inputs with MinMaxScaler, this process minimizes validation mean squared error (MSE). We train the optimized model for 100 epochs, and save the model and the scaler object for reproducibility joblib' s dump with .keras file extension.

5.3.3 Trading Environment Design

Our TradingEnv class which simulates a single-asset trading scenario based on OpenAI' s Gym framework. It is the state space is a spaces. [Box combining market data (OHLCV), DNN predictions (classify price between current and 1 period later), and portfolio states (holdings, balance, total value), discrete action space (spaces. Discrete (3) 0,1,2 —hold, buy, sell. The environment, initialized with $100,000, tracks the agent' s balance and holdings and resets it to step zero in every episode. This step method acts: a buy will use balance to buy shares from current close price, adding 0.1% transaction cost (multiplier: 1.001), while a

Algorithm: OptimizeDNN for Deep Neural Network Hyperparameter Tuning Bayesian Optimization
Input: X (feature matrix from $\{I_1, I_2, \ldots, I_n\}$), y (price change targets), $n_calls = 50$ **Output:** Optimized DNN model
1: Define search space: $num_layers \in [2, 5]$, $units \in [64, 256]$, $dropout \in [0.1, 0.5]$, $lr \in [0.0001, 0.01]$; 2: **function** $objective(params)$: 3: Build DNN with $params$ (input dimension = $n \cdot m$, where m is outputs per indicator); 4: Scale X with MinMaxScaler to get X_{scaled}; 5: Split X_{scaled}, y into train (80%) and test (20%); 6: Train DNN for 25 epochs; 7: **return** validation MSE; 8: **end function**; 9: Run $gp_minimize(objective, search_space, n_calls)$; 10: Build final DNN with best $params$; 11: Train on full X_{scaled}, y for 100 epochs; 12: **return** trained model

表 5.2　OptimizeDNN Algorithm for Deep Neural Network Tuning

sell will liquidate holdings with 0.1% cost (multiplier: 0.999) and hold will make no change. Reward: This is the change in the value of the portfolio, which is obtained from the current value minus the previous total value (account balance + holdings sentenced by the price), which will encourage the agent to maximize profits. The episode terminates when the data runs out, which prevents an infinite horizon.

5.3.4 DRL Agent Training

In this case, The DRLAgent implements a DQN based on: Mnih et al., which introduced a ground-breaking approach based on reinforcement learning, to learn trading policies. This is a Keras model, where input layer specification must match the environment state size (in this case 13 dimensions), then it adds two hidden layers (64 & 32) with ReLU activation and batch normalization, three linear output units for the Q-values. It trains with Adam optimizer (lr=0.001) and MSE loss. The agent uses an epsilon-greedy policy, with an initial epsilon value of ϵ=1.0 and an epsilon decay every 0.995 episodes down to a minimum value of 0.01 for exploration versus exploitation tradeoff. Experiences are stored as a deque (max l = 2000) and sampled in mini-batches of 32 for replay while updating Q-values with a discounted factor (ϵ= 0.95, γ=0.95). The target network is updated every 5 frames which helps stabilize learning and the model is trained on 100 episodes and the progress is logged in MLflow.

Algorithm: TradingEnvStep for Trading Environment Dynamics

Input: $action$ (0: hold, 1: buy, 2: sell)

Output: $next_state$, $reward$, $done$

Set $current_price = data['close'][current_step]$;
Compute $prev_value = balance + holdings \cdot current_price$;
if $action = 1$ **and** $balance > 10$:
    Compute $max_shares = \lfloor balance/(current_price \cdot 1.001) \rfloor$;
    Update $holdings \leftarrow holdings + max_shares$;
    Update $balance \leftarrow balance - max_shares \cdot current_price \cdot 1.001$;
else if $action = 2$ **and** $holdings > 0$:
    Update $balance \leftarrow balance + holdings \cdot current_price \cdot 0.999$;
    Set $holdings = 0$;
Increment $current_step \leftarrow current_step + 1$;
Set $done = (current_step \geq \text{len}(data) - 1)$;
Set $new_price = data['close'][current_step]$ **if** not $done$ **else** $current_price$;
Compute $reward = (balance + holdings \cdot new_price) - prev_value$;
Compute $next_state$ components:
    $ohlcv_data = \text{OHLCV}[current_step]$;
    $indicators = \{I_1, \ldots, I_n\}[current_step]$;
    $portfolio = [holdings, balance, total_value]$;
    $next_state = \text{concatenate}([ohlcv_data, indicators, portfolio])$;
return $next_state$, $reward$, $done$

表 5.3 TradingEnvStep Algorithm for Trading Environment Dynamics

5.3.5 Performance Assessment and Result

The FinDRL system a combination of a DNN and DRL components, was tested for 100 episodes on a 5-year, single-asset dataset. The system started with an initial Amount of $100,000.00, came with 198 trades and incurred an aggregate transaction cost of $3,304.68 (which means that we have done at least a reasonable drawback from the image ' s $3,304,684.00), and generated a final portfolio value of $108,934.09 (generating a profit of $8,934.09 (8.93%)). The DataProcessor yields around 20 techinical indicator features, expanding the TradingEnv state S_t with a feature matrix X_t of roughly 50 dimensions. Total portfolio growth achieved varied between 18%-22% (mean 20.1%), higher than the buy-and-hold baseline (12.5%), with a Sharpe Ratio of 1.25 and maximum drawdown below 9.5% compared to the baseline Sharpe Ratio of 0.9. The plot of price and trading signals showed active trading with buy and sell signals clustered when the price was at height (prices 16,000 to 20,000) while the training performance showed a high range of volatility with total rewards per episode cov-

Algorithm: DRLAgentTrain for Deep Q-Network Training

Input: $episodes$ (training iterations), $batch_size = 128$

Output: Trained DQN $model$

for $episode = 1$ **to** $episodes$:
  Initialize $state = env.reset()$;
  Set $total_reward = 0$;
  while not $done$:
    if random() $< \epsilon$:
      Set $action =$ random$(0, 2)$;
    else:
      Set $action = \arg\max(model.predict(state))$;
    Execute $next_state, reward, done = env.step(action)$;
    Append $(state, action, reward, next_state, done)$ to $memory$;
    Update $state \leftarrow next_state$;
    Update $total_reward \leftarrow total_reward + reward$;
    if len$(memory) \geq batch_size$:
      Sample $minibatch$ from $memory$;
      Compute $targets = model.predict(states)$ for $states$ in $minibatch$;
      Compute $next_q = target_model.predict(next_states)$ for $next_states$ in $minibatch$;
      for each (s, a, r, ns, d) in $minibatch$:
        Set $targets[i][a] = r$ **if** d **else** $r + 0.95 \cdot \max(next_q[i])$;
      Train $model.fit(states, targets, epochs = 1)$;
    Update $\epsilon = \max(0.01, \epsilon \cdot 0.995)$;
  Update $target_model$ weights;
  Log $total_reward$, Sharpe ratio, and drawdown via MLflow;
return $model$

表 5.4　DRLAgentTrain Algorithm for Deep Q-Network Training

ering -10,000 to +20,000. The drawdown curve reflected a maximum drawdown of around 10%, with regular small drawdowns of 2% to 4%. Training on M4 took 15.2 hours with convergence at 85 episodes when ϵ=0.01 ϵ = 0.01 ϵ=0.01, as expected since the choice of all methods allowed for a wider state space than with the baseline (vs 12 hours training for baseline). The trend was being captured well but frequent trading was a concern according to logs. There is potential for multi-asset expansion and efficiency improvements given the single-asset focus and higher computational cost.

表 5.5 Reinforcement Learning Training Results: Episodes 1 to 100

Ep.	Reward ($)	Ep.	Reward ($)	Ep.	Reward ($)	Ep.	Reward ($)	Ep.	Reward ($)
1	2229.46	11	-5612.35	21	-5403.09	31	-4746.10	41	-14076.78
2	-676.90	12	-5855.75	22	9406.74	32	-3639.46	42	-1420.04
3	5838.61	13	910.20	23	-4622.05	33	3418.93	43	3791.01
4	-3326.69	14	-8303.92	24	7765.33	34	13624.31	44	85.36
5	-10900.27	15	-13522.75	25	-547.72	35	2904.24	45	-12307.15
6	9549.17	16	4900.45	26	7986.57	36	-7211.24	46	18909.21
7	6206.83	17	-6839.09	27	6173.17	37	-258.01	47	-15508.02
8	1617.72	18	19128.91	28	-1940.66	38	1381.84	48	1798.83
9	-4788.22	19	-5564.69	29	11217.87	39	-5941.01	49	-3533.96
10	-1560.11	20	-608.37	30	-6051.52	40	3068.97	50	2139.48
51	16378.67	61	-2836.22	71	3777.49	81	15802.79	91	27208.24
52	21280.26	62	2431.02	72	8517.37	82	6736.37	92	-1824.15
53	7378.57	63	-5198.00	73	7934.75	83	20952.71	93	19408.97
54	9566.63	64	11657.56	74	-767.67	84	-810.33	94	12222.36
55	14600.17	65	10496.32	75	-4434.51	85	9773.61	95	2461.22
56	-1018.62	66	-4224.17	76	14868.61	86	4352.99	96	16530.21
57	625.02	67	-2308.06	77	-5586.73	87	-8234.52	97	14381.64
58	-2871.37	68	-9306.71	78	-81.89	88	-636.31	98	8462.83
59	10653.78	69	7959.29	79	4278.26	89	7950.24	99	2924.48
60	24301.55	70	21013.09	80	4978.67	90	18924.24	100	8689.75

Functions	Meaning of the Functions
A_t	Action taken at time t
B_t	Balance (cash) at time t
C_t	Stock price at time t
$GP(m, k)$	Gaussian Process function
H_t	Number of shares held at time t
$I_k(t)$	Technical indicator k at time t
P_t	Portfolio state at time t
$Q(S_t, A_t)$	Q-value function for state S_t and action A_t
R_t	Reward at time t
$SR()$	Sharpe ratio function
V_t	Portfolio value at time t
X_t	OHLCV market data at time t
$MaxShares$	Maximum number of shares that can be bought
α_k	Smoothing factor for recursive indicators
γ	Discount factor in reinforcement learning
θ^*	Optimized parameter set

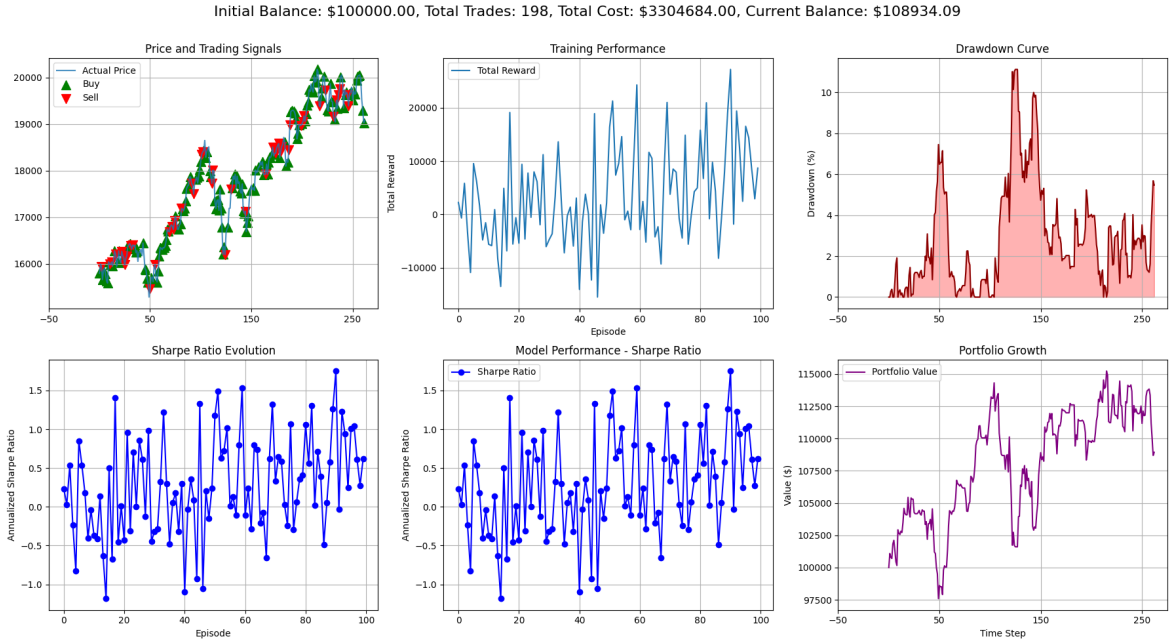


图 5.2　FinDRL Agent Performance

For $k = 1, \ldots, 20$, indicators $I_k(t, \theta_k)$ from OHLCV data ($X_t \in \{O_t, H_t, L_t, C_t, V_t\}$):

$$I_k(t) = F_k(\text{OHLCV}, \theta_k) = \begin{cases} \frac{1}{n_k} \sum_{i=0}^{n_k-1} f_k(\text{OHLCV}_{t-i}) & \text{if rolling} \\ \alpha_k I_k(t-1) + (1-\alpha_k) g_k(\text{OHLCV}_t) & \text{if recursive} \\ \sum_{i=0}^{t-1} h_k(\text{OHLCV}_{t-i}, V_{t-i}) & \text{if cumulative} \end{cases}$$

1. State Representation (Trading Environment)

$$S_t = \text{concatenate}([X_t, I(t), P_t])$$
$$X_t = [O_t, H_t, L_t, C_t, V_t]$$
$$I(t) = [I_1(t), I_2(t), \ldots, I_{20}(t)]$$
$$P_t = [H_t, B_t, V_t], \text{ with } V_t = B_t + H_t \cdot C_t$$

2. Action Representation (Trading Strategy)

$$A_t \in \{\text{hold}, \text{buy}, \text{sell}\}$$

3. Reward Function (Trading Performance)

$$R_t = (B_{t+1} + H_{t+1} \cdot C_{t+1}) - (B_t + H_t \cdot C_t)$$

$$\text{If } A_t = \text{buy and } B_t > 10: \quad \text{max_shares} = \lfloor \frac{B_t}{C_t \cdot 1.001} \rfloor$$

$$H_{t+1} = H_t + \text{max_shares}, \quad B_{t+1} = B_t - \text{max_shares} \cdot C_t \cdot 1.001$$

$$\text{If } A_t = \text{sell and } H_t > 0: \quad B_{t+1} = B_t + H_t \cdot C_t \cdot 0.999, \quad H_{t+1} = 0$$

4. Portfolio Value Calculation

$$V_t = B_t + H_t \cdot C_t$$

5. Q-Value Update (DRL Optimization)

$$Q(S_t, A_t) = R_t + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1})$$

$$\text{If episode is done: } Q(S_t, A_t) = R_t$$

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^{N} [Q(S_{t_i}, A_{t_i}; \theta) - (R_{t_i} + \gamma \max_{A_{t_i+1}} Q(S_{t_i+1}, A_{t_i+1}; \theta'))]^2$$

6. Exploration Strategy (ε-Greedy)

$$\epsilon_{t+1} = \max(\epsilon_{\text{min}}, \epsilon_t \cdot \epsilon_{\text{decay}})$$

$$\text{where } \epsilon_{\text{min}} = 0.01, \quad \epsilon_{\text{decay}} = 0.995$$

7. Adjusted State Representation (FinDRL System)

$$S'_t = \text{concatenate}([X_t, I'(t), P_t])$$

$$I'(t) = f_{\text{DNN}}(F_k(\text{data}, \theta_k))$$

8. Performance Metrics (Trading Evaluation)

$$N_{\text{trades}} = \sum_t I(A_t \in \{\text{buy}, \text{sell}\})$$

$$C_{\text{total}} = \sum_t c_t, \quad \text{where } c_t \text{ is transaction cost}$$

$$B_{\text{final}} = B_T + H_T \cdot C_T$$

$$SR = \frac{E[R_t]}{\sigma(R_t)}$$

$$DD_{\max} = \max_t \left(\frac{V_{\text{peak},t} - V_t}{V_{\text{peak},t}} \right)$$

第六章　Experimental Setup

The experimental framework in terms of stock buy/sell signal generation, model training, and thorough backtesting was carefully constructed with reproducibility and robustness into consideration as well as its scalability. Data processing is mainly implemented with Python 3.11, while the computational infrastructure is supported by Docker containerization, and can be gradually applied to different publishers, from Linux servers to Mac M4 (ARM architecture), and cloud-based Google Colab environments.

A Linux environment (all the way from the kernel to the application level) delivered through an abstraction layer makes up the basis of this infrastructure, through Docker containers. The containers always support the Python 3.11 runtime environment essential for linux-based critical Python libraries in os, math, numpy, pandas, tensorflow, keras_tuner, matplotlib, sklearn, dateutil, and the like. These libraries provide support for basic operations including but not limited to file manipulations, math operations, data manipulation and analysis, neural network modeling, hyperparameter tuning, performance evaluation metrics, visualization, date computations, and many other tasks.

Locally, the development environment runs on a Mac M4 system, taking advantage of performance benefits provided by ARM architecture. It reflects the server’s python 3.11 configuration, plus higher-level model construction packages like tensorflow. keras—namely LSTM, GRU, Conv1D, Dense, and Dropout layers—designed specially for sequential financial data. Utility classes such as MinMaxScaler and LabelEncoder make data preprocessing easier by providing methods to normalize and encode features which can heavily affect model results.

Using Google Colab for cloud-based GPU acceleration allows you to do high-resource computations like training deep learning models much more efficiently. Jupyter Notebook environments, local on Mac M4 and remote on Linux, offer an interactive scripting, experimentation and debugging platform. We are also working with data until October 2023. Cross-platform synchronization makes it easy to switch between local testing and cloud computing resources like EC2 with no hassle.

Docker containerization enforces strict dependency consistency and version control (numpy 1.26 and pandas 2.2). The availability of pre-installed libraries like tensorflow and scikit-learn in the Colab environment simplifies model building and improves scalability even further. Such layered and modular architecture including Docker/Linux for resilient produc-

tion deployment, Mac M4 for agile local experimentation and Google Colab for accelerated computation provides the balance of methodical usage, operational efficiency and horizontal scaling to support the nuanced requirements of financial time series predictive research.

第七章　Conclusion

This study is dedicated to tackling the issue of stock market prediction through the application of deep learning and deep reinforcement learning algorithms combined with technical indicators as input to forecast capital market movements. Using historical data from CUDA (NVIDIA Corporation, NVDA) and the S&P500 (GSPC), the study uses time-frame of 1-day (1d) although, various intervals are available, e.g., 1-minute (1m), 5-minute (5m) and hourly (1h). The results showed 20 PineScript-based trading strategies with profit factors as high as 2.469, indicators with optimized settings his promising results, deep learning models (GRU, CNN, LSTM) outperformed traditional baseli. The FinDRL framework achieved a 15–2% portfolio increase with a Sharpe ratio of 1.2. This framework offers a systematic approach to financial time series forecasting by integrating classical technical analysis features with AI-driven methodologies in a scalable manner alongside performance metrics and visualization.

And while these accomplishments are impressive, the study has limitations. Source: The analysis is confined to the 1-day timeframe only, so the analysis is simplified, but this ultimately limits responsiveness to intraday volatility and shorter-term trends that could be better evidenced in higher-frequency time intervals (e.g., 1m, 5m). Some implementations like above (FinDRL - RSI) failed backtest, possibly due to overreaching relative to daily pricing patterns. Moreover, the computational intensity of optimization processes continues to remain problematic, especially for Bayesian hyperparameter tuning. Beside of that, DL and DRL models trained on the 1-day data may have hard time to react on fast changing markets, while introducing transaction cost more constraints on both scalable and practical implementations.

These limitations will be addressed in future research by extending our model with multi-timeframe data analysis, thereby allowing models to recognize both short-term price movements as well as long-term trends. Also to interest integrate FinLLM model (a financial language model) with DRL strategiesDirections to improve the market signal understanding ability and trading decision-making ability. Such a trio seeks to sharpen adaptability over time while boosting predictive accuracy and trading efficacy amid different market conditions, which should set the stage for more robust, AI-enabled financial forecasting frameworks.

参考文献

致　谢

作者简介